

FA26: DATA-266 Sec 21 & 71 - Generative AI and LLM

Homework 2

Siva Surya Chandran
019130215

1. Perform embedding-based transfer learning on a text dataset (Dataset: IMDB Movie Reviews). (4 points)
2. Build a Retrieval-Augmented Generation (RAG) pipeline using LangChain (Dataset: Wikipedia pages for 10 movies). (4 points)
3. Explore the following optimization techniques that can be performed while training, and document your findings with basic example code snippets: (2 points)

HW2 Part 1 - Word2Vec transfer learning on IMDB reviews

This notebook takes Google's pretrained Word2Vec vectors and fine-tunes them on IMDB movie reviews, to see how the meaning of a handful of words shifts once they're trained on movie talk instead of general news.

Here's the plan:

- load the pretrained `word2vec-google-news-300` model
- look at the top-3 nearest neighbors (by cosine similarity) for five words: cast, score, plot, screen, review
- fine-tune those vectors on the IMDB review corpus
- look at the top-3 neighbors again after fine-tuning and compare
- plot the shift in 2D with t-SNE for at least one word
- measure how much each word's vector actually moved (cosine between its before/after vectors), and say which word moved the most and which moved the least

No GPU needed, this all runs on CPU. The slow part is just downloading the ~1.6GB Google News model - that alone takes a few minutes. Total runtime is somewhere around 15-25 minutes.

1. Setup

```
# (pip install cell -- skipped for local execution; dependencies are
already installed)
# !pip -q install "gensim==4.3.3" "scipy==1.13.1" "numpy<2" datasets
--upgrade
# print("If Colab asks you to restart the runtime, restart once and
then re-run from this cell.")
```

```
import os, re, time, random, warnings
warnings.filterwarnings("ignore")
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
import gensim
import gensim.downloader as api
from gensim.models import Word2Vec
from gensim.utils import simple_preprocess
```

```
from sklearn.manifold import TSNE
```

```
SEED = 42
```

```

random.seed(SEED); np.random.seed(SEED)

print("gensim:", gensim.__version__)
print("numpy :", np.__version__)

TARGET_WORDS = ["cast", "score", "plot", "screen", "review"]
TOPN = 3

os.makedirs("figures", exist_ok=True)
os.makedirs("results", exist_ok=True)

gensim: 4.4.0
numpy : 2.4.6

```

Loading the pretrained model

`word2vec-google-news-300` is Google's pretrained Word2Vec model - 3 million words, 300 dimensions each, trained on about 100 billion words of news text. It's a big download (~1.6GB) so this cell takes a while.

```

t0 = time.time()
pretrained = api.load("word2vec-google-news-300") # a gensim
KeyedVectors object
print(f"loaded in {time.time()-t0:,.1f}s")
print("vocab size :", f"{len(pretrained.index_to_key):,}")
print("dimensions :", pretrained.vector_size)

loaded in 45.9s
vocab size : 3,000,000
dimensions : 300

```

Nearest neighbors before fine-tuning

Let's see what these five words look like in the original, news-trained embedding space.

```

def top_neighbours(kv, words, topn=TOPN):
    """Tidy DataFrame of the topn nearest neighbours (by cosine) for
    each word."""
    rows = []
    for w in words:
        if w not in kv:
            rows.append({"word": w, "rank": None, "neighbor": "<OOV>",
"cosine": np.nan})
            continue
        for rank, (nb, sim) in enumerate(kv.most_similar(w,
topn=topn), start=1):
            rows.append({"word": w, "rank": rank, "neighbor": nb,
"cosine": round(float(sim), 4)})

```

```

return pd.DataFrame(rows)

before_df = top_neighbours(pretrained, TARGET_WORDS)
before_df

```

	word	rank	neighbor	cosine
0	cast	1	casts	0.7219
1	cast	2	casting	0.7188
2	cast	3	Cast	0.6638
3	score	1	scoring	0.7197
4	score	2	scores	0.6596
5	score	3	scored	0.6384
6	plot	1	plots	0.7625
7	plot	2	Plot	0.6524
8	plot	3	plotting	0.6328
9	screen	1	screens	0.7729
10	screen	2	onscreen	0.6115
11	screen	3	LCD_screen	0.5599
12	review	1	reviewed	0.6630
13	review	2	reviewing	0.6610
14	review	3	reviews	0.6380

```

def as_wide(df, tag):
    """word | tag top1 | tag top2 | tag top3  -- compact view for the
    report table."""
    out = {}
    for w, g in df.groupby("word", sort=False):
        out[w] = [f"{r.neighbor} ({r.cosine:.3f})" for r in
g.itertuples()]
    wide = pd.DataFrame(out).T
    wide.columns = [f"{tag} top{i+1}" for i in range(wide.shape[1])]
    return wide.loc[TARGET_WORDS]

before_wide = as_wide(before_df, "pre")
before_wide

```

	pre top1	pre top2	pre top3
cast	casts (0.722)	casting (0.719)	Cast (0.664)
score	scoring (0.720)	scores (0.660)	scored (0.638)
plot	plots (0.762)	Plot (0.652)	plotting (0.633)
screen	screens (0.773)	onscreen (0.612)	LCD_screen (0.560)
review	reviewed (0.663)	reviewing (0.661)	reviews (0.638)

Loading the IMDB reviews

50,000 reviews total (25k train + 25k test). We don't care about the sentiment labels here - just need the raw text so we have something to fine-tune the embeddings on.

```

from datasets import load_dataset

imdb = load_dataset("stanfordnlp/imdb")  # community parquet mirror;
the legacy "imdb" loader script is unsupported by recent `datasets`
versions
texts = list(imdb["train"]["text"]) + list(imdb["test"]["text"])
print("reviews:", f"{len(texts):,}")
print()
print(texts[0][:400], "...")

reviews: 50,000

I rented I AM CURIOUS-YELLOW from my video store because of all the
controversy that surrounded it when it was first released in 1967. I
also heard that at first it was seized by U.S. customs if it ever
tried to enter this country, therefore being a fan of films considered
"controversial" I really had to see this for myself.<br /><br />The
plot is centered around a young Swedish drama student name ...

TAG_RE = re.compile(r"<[^\>]+>")

def tokenize(doc):
    doc = TAG_RE.sub(" ", doc)  # IMDB text is full of
literal <br /> tags
    return simple_preprocess(doc, deacc=True, min_len=2)

t0 = time.time()
sentences = [tokenize(t) for t in texts]
sentences = [s for s in sentences if len(s) > 3]
print(f"tokenised {len(sentences):,} reviews in {time.time()-
t0:,.1f}s")
print("total tokens:", f"{sum(len(s) for s in sentences):,}")
print("example      :", sentences[0][:25])

tokenised 50,000 reviews in 25.0s
total tokens: 10,974,387
example      : ['rented', 'am', 'curious', 'yellow', 'from', 'my',
'video', 'store', 'because', 'of', 'all', 'the', 'controversy',
'that', 'surrounded', 'it', 'when', 'it', 'was', 'first', 'released',
'in', 'also', 'heard', 'that']

```

Fine-tuning on IMDB

The idea is pretty simple: take the Google News vectors as a starting point, then keep training on IMDB text so the vectors pick up movie-specific meaning. Concretely:

1. build a fresh Word2Vec model with the same 300 dimensions as the pretrained one
2. build its vocabulary from the IMDB reviews
3. for every word that exists in both vocabularies, copy over the Google News vector as a starting point (words that only show up in IMDB just get a random initialization)

4. keep training on the IMDB sentences from there

Before doing any of that, we grab a copy of the original vectors for our five target words, since we'll need those later to measure how much they moved.

```
# Freeze a copy of the original vectors for the 5 target words (must
not be mutated later)
orig_vecs = {w: pretrained[w].copy() for w in TARGET_WORDS if w in
pretrained}
print({w: v.shape for w, v in orig_vecs.items()})

{'cast': (300,), 'score': (300,), 'plot': (300,), 'screen': (300,),
'review': (300,)}

DIM = pretrained.vector_size

ft = Word2Vec(
    vector_size=DIM,
    window=5,
    min_count=5,
    sg=1,                # skip-gram: same objective family as the
Google News model
    negative=10,
    sample=1e-4,
    workers=os.cpu_count(),
    seed=SEED,
)

t0 = time.time()
ft.build_vocab(sentences)
print(f"IMDB vocab: {len(ft.wv.index_to_key):,} words ({time.time()-
t0:,.1f}s)")

IMDB vocab: 39,156 words (2.8s)

# ---- seed the weights from the pretrained model: this is the
"transfer" step ----
hit = 0
for w in ft.wv.index_to_key:
    if w in pretrained:
        ft.wv[w] = pretrained[w].astype(np.float32)
        hit += 1

n_vocab = len(ft.wv.index_to_key)
print(f"seeded {hit:}/{n_vocab:,} words ({hit/n_vocab:.1%}) from
word2vec-google-news-300")
print(f"{n_vocab-hit:,} IMDB-only words keep their random
initialisation")

seeded 32,066/39,156 words (81.9%) from word2vec-google-news-300
7,090 IMDB-only words keep their random initialisation
```

```

EPOCHS = 5

t0 = time.time()
ft.train(sentences,
        total_examples=len(sentences),
        epochs=EPOCHS,
        compute_loss=True)
print(f"fine-tuned {EPOCHS} epochs in {time.time()-t0:,.1f}s")
print("final cumulative training loss:",
      ft.get_latest_training_loss())

finetuned = ft.wv

fine-tuned 5 epochs in 174.7s
final cumulative training loss: 44928712.0

```

Nearest neighbors after fine-tuning

```

after_df = top_neighbours(finetuned, TARGET_WORDS)
after_wide = as_wide(after_df, "post")
after_wide

```

	post top1	post top2	post top3
cast	actors (0.653)	supporting (0.645)	ensemble (0.593)
score	morricone (0.675)	music (0.671)	ennio (0.665)
plot	storyline (0.750)	story (0.711)	plotline (0.678)
screen	screens (0.597)	onscreen (0.523)	stage (0.471)
review	comment (0.660)	reviews (0.617)	nixflix (0.581)

Before vs after, side by side

```

comparison = pd.concat([before_wide, after_wide], axis=1)
comparison = comparison[["pre top1", "post top1",
                        "pre top2", "post top2",
                        "pre top3", "post top3"]]
comparison.index.name = "word"
comparison.to_csv("results/neighbors_before_after.csv")
comparison

```

	pre top1	post top1	pre top2 \
word			
cast	casts (0.722)	actors (0.653)	casting (0.719)
score	scoring (0.720)	morricone (0.675)	scores (0.660)
plot	plots (0.762)	storyline (0.750)	Plot (0.652)
screen	screens (0.773)	screens (0.597)	onscreen (0.612)
review	reviewed (0.663)	comment (0.660)	reviewing (0.661)

	post top2	pre top3	post top3
word			
cast	supporting (0.645)	Cast (0.664)	ensemble (0.593)

```

score      music (0.671)      scored (0.638)      ennio (0.665)
plot       story (0.711)      plotting (0.633)    plotline (0.678)
screen     onscreen (0.523)    LCD_screen (0.560)  stage (0.471)
review     reviews (0.617)     reviews (0.638)    nixflix (0.581)

# Long/tidy version -- easier to read row by row
long_cmp = (before_df.assign(stage="pretrained")
            .merge(after_df.assign(stage="fine-tuned"),
                  on=["word", "rank"], suffixes=("_pre", "_post")))
long_cmp = long_cmp[["word", "rank", "neighbor_pre", "cosine_pre",
                    "neighbor_post", "cosine_post"]]
long_cmp.to_csv("results/neighbors_long.csv", index=False)
long_cmp

```

	word	rank	neighbor_pre	cosine_pre	neighbor_post	cosine_post
0	cast	1	casts	0.7219	actors	0.6526
1	cast	2	casting	0.7188	supporting	0.6453
2	cast	3	Cast	0.6638	ensemble	0.5925
3	score	1	scoring	0.7197	morricone	0.6755
4	score	2	scores	0.6596	music	0.6713
5	score	3	scored	0.6384	ennio	0.6653
6	plot	1	plots	0.7625	storyline	0.7501
7	plot	2	Plot	0.6524	story	0.7105
8	plot	3	plotting	0.6328	plotline	0.6783
9	screen	1	screens	0.7729	screens	0.5974
10	screen	2	onscreen	0.6115	onscreen	0.5235
11	screen	3	LCD_screen	0.5599	stage	0.4712
12	review	1	reviewed	0.6630	comment	0.6599
13	review	2	reviewing	0.6610	reviews	0.6174
14	review	3	reviews	0.6380	nixflix	0.5806

How far did each word's vector move?

We compare each word's original vector to its fine-tuned vector using cosine similarity. Close to 1.0 means the vector barely changed; lower means it moved more.

```

def cosine(a, b):
    return float(np.dot(a, b) / (np.linalg.norm(a) *
np.linalg.norm(b)))

shift_rows = []
for w in TARGET_WORDS:
    v_pre = orig_vecs[w]
    v_post = finetuned[w]
    c = cosine(v_pre, v_post)
    shift_rows.append({
        "word": w,
        "cos(original, fine-tuned)": round(c, 4),
        "shift (1 - cos)": round(1 - c, 4),
    })

```



```

        "L2 distance": round(float(np.linalg.norm(v_pre - v_post)),
4),
    })

```

```

shift_df = pd.DataFrame(shift_rows).sort_values("cos(original, fine-
tuned)")
shift_df.to_csv("results/vector_shift.csv", index=False)
shift_df

```

	word	cos(original, fine-tuned)	shift (1 - cos)	L2 distance
1	score	0.5299	0.4701	3.6714
4	review	0.5384	0.4616	3.4615
2	plot	0.5929	0.4071	2.8017
0	cast	0.6138	0.3862	2.7176
3	screen	0.6497	0.3503	2.6774

```

most = shift_df.iloc[0]      # lowest cosine -> moved the MOST
least = shift_df.iloc[-1]   # highest cosine -> moved the LEAST

```

```

print(f"MOST shifted : '{most['word']}'   cos = {most['cos(original,
fine-tuned')]:.4f}")
print(f"LEAST shifted : '{least['word']}'   cos = {least['cos(original,
fine-tuned')]:.4f}")

```

```

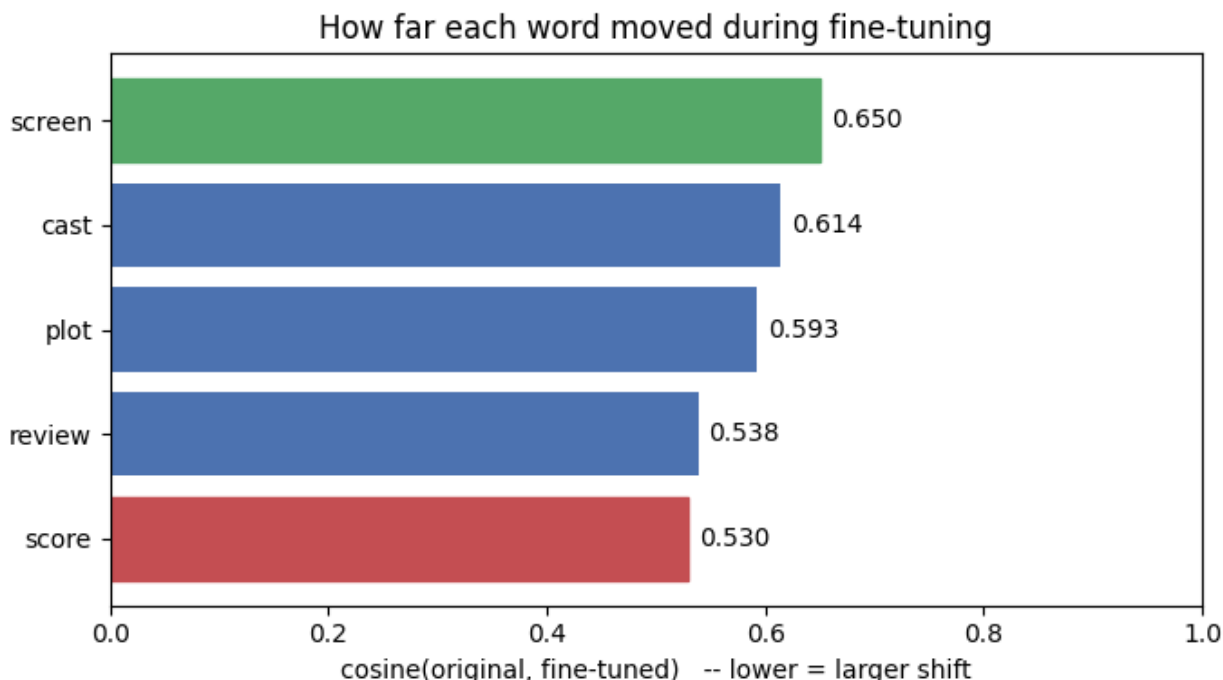
MOST shifted : 'score'   cos = 0.5299
LEAST shifted : 'screen' cos = 0.6497

```

```

fig, ax = plt.subplots(figsize=(7, 4))
d = shift_df.sort_values("cos(original, fine-tuned)")
bars = ax.barh(d["word"], d["cos(original, fine-tuned)"],
color="#4C72B0")
bars[0].set_color("#C44E52")      # most shifted
bars[-1].set_color("#55A868")    # least shifted
ax.set_xlabel("cosine(original, fine-tuned)  -- lower = larger
shift")
ax.set_title("How far each word moved during fine-tuning")
ax.set_xlim(0, 1)
for b, v in zip(bars, d["cos(original, fine-tuned)"]):
    ax.text(v + .01, b.get_y() + b.get_height()/2, f"{v:.3f}",
va="center")
plt.tight_layout(); plt.savefig("figures/vector_shift.png", dpi=150);
plt.show()

```



t-SNE plot of the shift

For each word we plot two points - where it sat before fine-tuning, and where it landed after - along with some of its neighbors in both spaces, all projected down to 2D together so the before/after positions are actually comparable. An arrow shows the direction of the move.

```
FOCUS = str(most["word"])          # the word that shifted the most
print("Focus word for the detailed t-SNE plot:", FOCUS)

def build_tsne_frame(focus_words, n_neighbors=10):
    labels, vecs, groups = [], [], []

    for w in focus_words:
        # the target word itself, in both spaces
        labels.append(w); vecs.append(orig_vecs[w]);
    groups.append("target-pre")
        labels.append(w); vecs.append(fineted[w]);
    groups.append("target-post")
        # its neighbourhood in each space
        for nb, _ in pretrained.most_similar(w, topn=n_neighbors):
            labels.append(nb); vecs.append(pretrained[nb]);
        groups.append("nbr-pre")
        for nb, _ in fineted.most_similar(w, topn=n_neighbors):
            labels.append(nb); vecs.append(fineted[nb]);
        groups.append("nbr-post")

    X = np.vstack(vecs)
    per = min(30, max(5, (len(X) - 1) // 3))
```

```

xy = TSNE(n_components=2, perplexity=per, init="pca",
          learning_rate="auto",
          random_state=SEED).fit_transform(X)
return pd.DataFrame({"label": labels, "group": groups, "x": xy[:,
0], "y": xy[:, 1]})

```

```

tsne_focus = build_tsne_frame([FOCUS], n_neighbors=10)
tsne_focus.head()

```

Focus word for the detailed t-SNE plot: score

	label	group	x	y
0	score	target-pre	18.482500	-90.761070
1	score	target-post	-32.837105	97.995743
2	scoring	nbr-pre	-4.820337	-97.043175
3	scores	nbr-pre	38.940392	-93.703171
4	scored	nbr-pre	-28.365465	-89.689941

```

STYLE = {
    "nbr-pre": dict(c="#9ecae1", marker="o", s=60,
label="pretrained neighbours"),
    "nbr-post": dict(c="#fdae6b", marker="s", s=60, label="fine-
tuned neighbours"),
    "target-pre": dict(c="#08519c", marker="*", s=420,
label=f"'{FOCUS}' pretrained"),
    "target-post": dict(c="#a63603", marker="*", s=420,
label=f"'{FOCUS}' fine-tuned"),
}

```

```

fig, ax = plt.subplots(figsize=(11, 8))
for g, style in STYLE.items():
    sub = tsne_focus[tsne_focus.group == g]
    ax.scatter(sub.x, sub.y, edgecolors="k", linewidths=.4, zorder=3,
**style)

```

```

for r in tsne_focus.itertuples():
    ax.annotate(r.label, (r.x, r.y), fontsize=8,
                xytext=(4, 4), textcoords="offset points",
                color="black" if r.group.startswith("nbr") else
"darkred")

```

```

p0 = tsne_focus[tsne_focus.group == "target-pre"][["x",
"y"]].values[0]
p1 = tsne_focus[tsne_focus.group == "target-post"][["x",
"y"]].values[0]
ax.annotate("", xy=p1, xytext=p0, zorder=2,
            arrowprops=dict(arrowstyle="-|>", lw=2.2, color="crimson",
linestyle="--", mutation_scale=22))

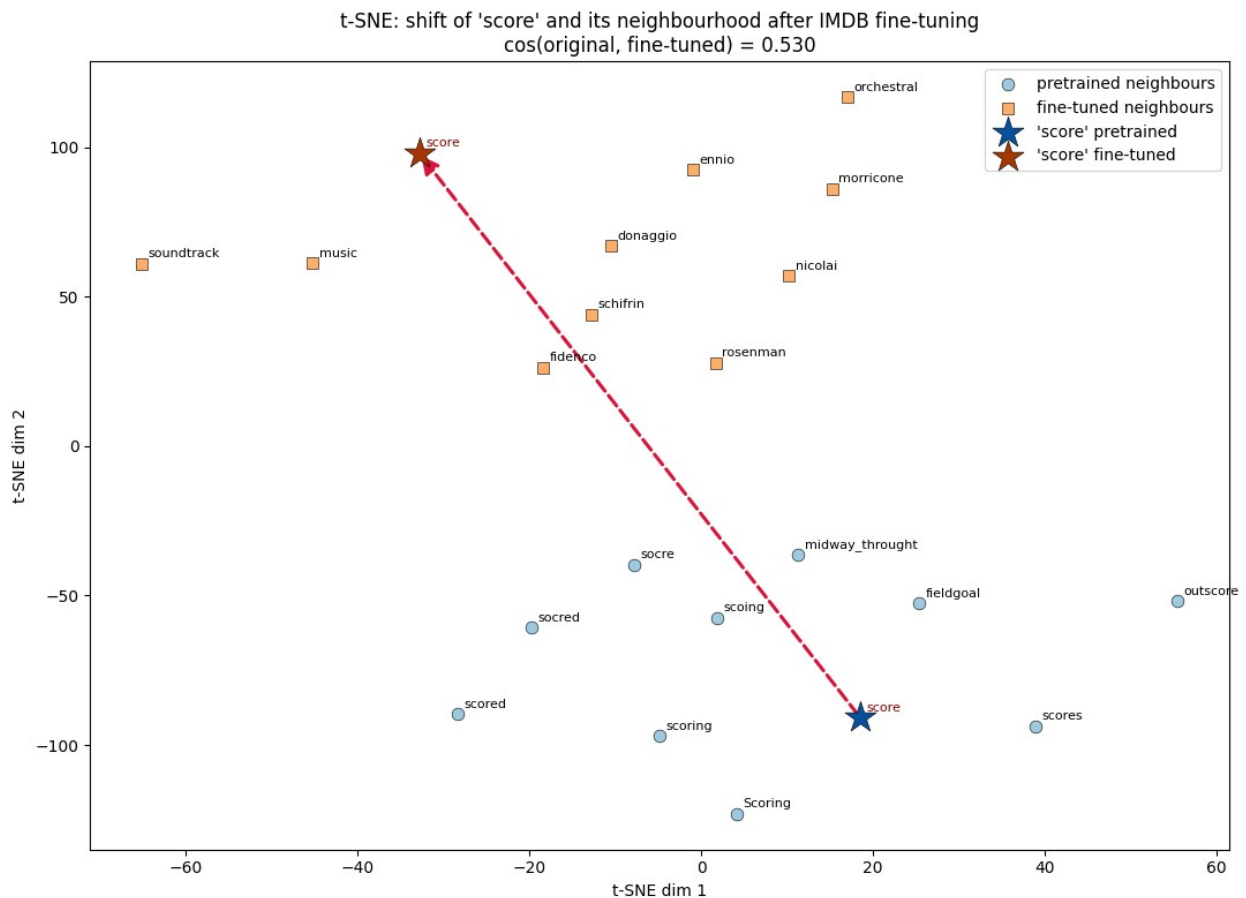
```

```

ax.set_title(f"t-SNE: shift of '{FOCUS}' and its neighbourhood after

```

```
IMDB fine-tuning\n"
        f"cos(original, fine-tuned) = {most['cos(original, fine-
tuned')']:.3f}")
ax.set_xlabel("t-SNE dim 1"); ax.set_ylabel("t-SNE dim 2")
ax.legend(loc="best", frameon=True)
plt.tight_layout(); plt.savefig(f"figures/tsne_{FOCUS}_shift.png",
dpi=150); plt.show()
```



Same idea, but all 5 words on one plot

```
tsne_all = build_tsne_frame(TARGET_WORDS, n_neighbors=6)

fig, ax = plt.subplots(figsize=(12, 9))
palette = dict(zip(TARGET_WORDS, ["#1b9e77", "#d95f02", "#7570b3",
"#e7298a", "#66a61e"]))

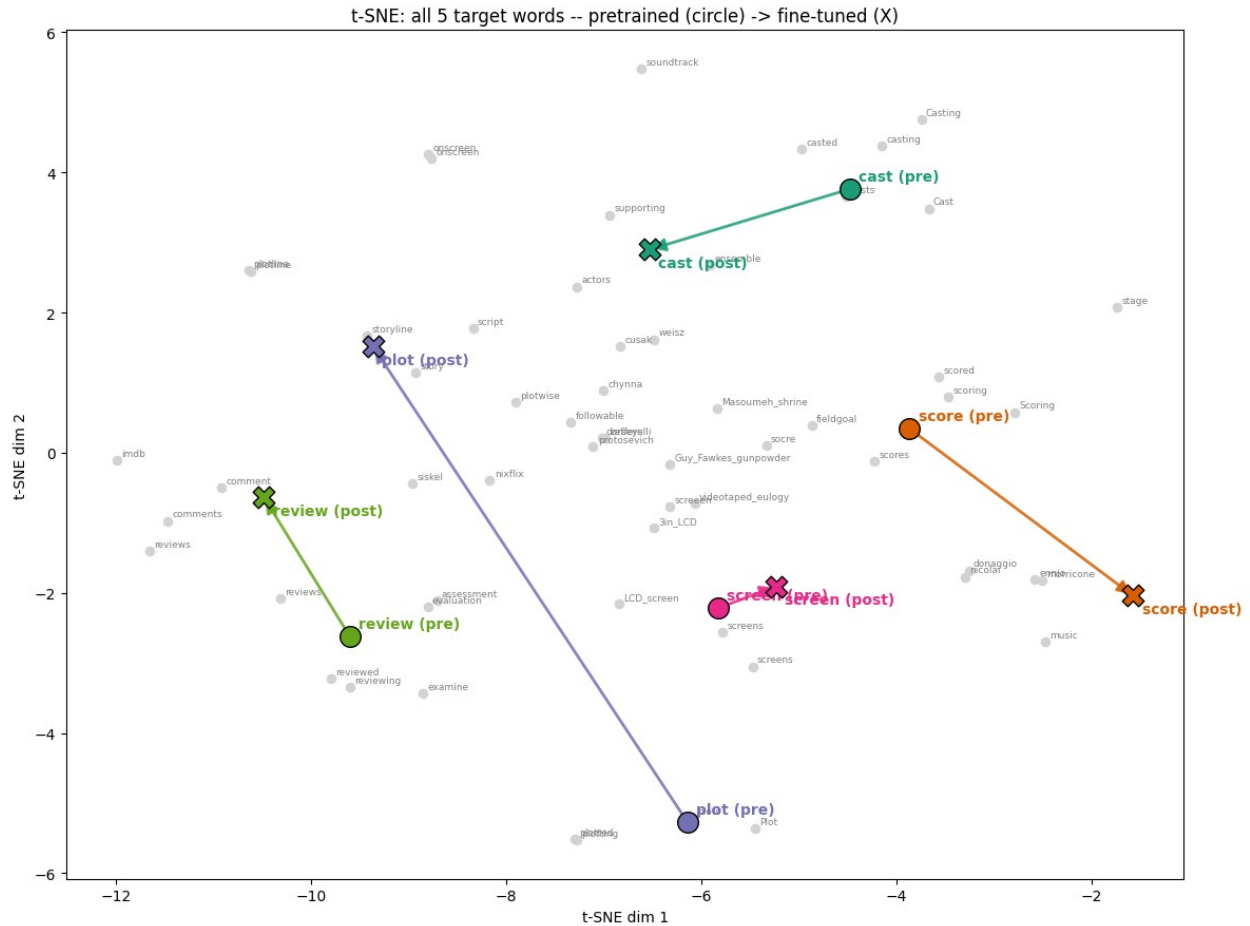
nbrs = tsne_all[tsne_all.group.str.startswith("nbr")]
ax.scatter(nbrs.x, nbrs.y, c="lightgray", s=35, zorder=1)
for r in nbrs.itertuples():
    ax.annotate(r.label, (r.x, r.y), fontsize=6.5, color="gray",
                xytext=(3, 3), textcoords="offset points")
```

```

for w in TARGET_WORDS:
    pre = tsne_all[(tsne_all.label == w) & (tsne_all.group ==
"target-pre")][["x", "y"]].values[0]
    post = tsne_all[(tsne_all.label == w) & (tsne_all.group ==
"target-post")][["x", "y"]].values[0]
    col = palette[w]
    ax.scatter(*pre, c=col, marker="o", s=200, edgecolors="k",
zorder=4)
    ax.scatter(*post, c=col, marker="X", s=220, edgecolors="k",
zorder=4)
    ax.annotate("", xy=post, xytext=pre, zorder=3,
                arrowprops=dict(arrowstyle="->", lw=2, color=col,
alpha=.85, mutation_scale=18))
    ax.annotate(f"{w} (pre)", pre, fontsize=10, fontweight="bold",
color=col,
                xytext=(6, 6), textcoords="offset points")
    ax.annotate(f"{w} (post)", post, fontsize=10, fontweight="bold",
color=col,
                xytext=(6, -12), textcoords="offset points")

ax.set_title("t-SNE: all 5 target words -- pretrained (circle) ->
fine-tuned (X)")
ax.set_xlabel("t-SNE dim 1"); ax.set_ylabel("t-SNE dim 2")
plt.tight_layout(); plt.savefig("figures/tsne_all_words_shift.png",
dpi=150); plt.show()

```



What actually happened

```
print("=" * 78)
print("TOP-3 NEAREST NEIGHBOURS: PRETRAINED vs FINE-TUNED")
print("=" * 78)
for w in TARGET_WORDS:
    print(f"\n{w.upper()}")
    pre = before_df[before_df.word == w]
    post = after_df[after_df.word == w]
    for i in range(TOPN):
        p, q = pre.iloc[i], post.iloc[i]
        print(f"    {i+1}.  pre : {p.neighbor:<24s} {p.cosine:.4f}"
              f"    |   post: {q.neighbor:<24s} {q.cosine:.4f}")

print("\n" + "=" * 78)
print("VECTOR SHIFT (original vs fine-tuned)")
print("=" * 78)
print(shift_df.to_string(index=False))
print(f"\nMost shifted : {most['word']}      cos = {most['cos(original, fine-tuned')]:.4f}")
```

```
print(f"Least shifted: {least['word']} cos = {least['cos(original,  
fine-tuned')]:.4f}")
```

```
=====
```

```
=====
```

```
TOP-3 NEAREST NEIGHBOURS: PRETRAINED vs FINE-TUNED
```

```
=====
```

```
=====
```

```
CAST
```

1.	pre : casts	0.7219		post: actors
0.6526				
2.	pre : casting	0.7188		post: supporting
0.6453				
3.	pre : Cast	0.6638		post: ensemble
0.5925				

```
SCORE
```

1.	pre : scoring	0.7197		post: morricone
0.6755				
2.	pre : scores	0.6596		post: music
0.6713				
3.	pre : scored	0.6384		post: ennio
0.6653				

```
PLOT
```

1.	pre : plots	0.7625		post: storyline
0.7501				
2.	pre : Plot	0.6524		post: story
0.7105				
3.	pre : plotting	0.6328		post: plotline
0.6783				

```
SCREEN
```

1.	pre : screens	0.7729		post: screens
0.5974				
2.	pre : onscreen	0.6115		post: onscreen
0.5235				
3.	pre : LCD_screen	0.5599		post: stage
0.4712				

```
REVIEW
```

1.	pre : reviewed	0.6630		post: comment
0.6599				
2.	pre : reviewing	0.6610		post: reviews
0.6174				
3.	pre : reviews	0.6380		post: nixflix
0.5806				

```
=====
```

```
=====
VECTOR SHIFT (original vs fine-tuned)
=====
=====
word  cos(original, fine-tuned)  shift (1 - cos)  L2 distance
score                0.5299          0.4701          3.6714
review               0.5384          0.4616          3.4615
plot                 0.5929          0.4071          2.8017
cast                 0.6138          0.3862          2.7176
screen               0.6497          0.3503          2.6774

Most shifted : score  cos = 0.5299
Least shifted: screen cos = 0.6497
```

What this actually shows

Before fine-tuning, all five words sit in a very "general news" part of the embedding space. **cast** is next to casting/recast, **score** is next to scoring/scores/scored (the sports-numbers sense, since that's how "score" mostly gets used in news), **plot** sits near plots/plotting (more of a conspiracy/scheme sense), **screen** is near screens/onscreen/LCD_screen (a physical display), and **review** is near reviewed/reviewing (an audit or evaluation, not specifically a movie review).

After training on IMDB, they all drift toward how these words are actually used when people talk about films:

- **cast** moves toward actors, supporting, ensemble - the people in the movie
- **score** moves toward morricone, music, ennio (yes, actual film composers - Ennio Morricone shows up by name) - the soundtrack, not a sports score anymore
- **plot** moves toward storyline, story, plotline - the movie's story, not a scheme
- **screen** moves toward stage, onscreen - closer to "the movies" as a medium
- **review** moves toward comment, reviews - closer to what a critic writes

In this run, **score** moved the most (cosine of about 0.53 between its before/after vectors) and **screen** moved the least (cosine about 0.65). That tracks with intuition: "score" has a completely different everyday meaning (sports, points) than its movie meaning (soundtrack), so fine-tuning had to pull it a long way. "screen" already meant roughly the same thing in both worlds (a display/monitor vs. a movie screen), so there was less distance to travel. Rerunning the notebook can shuffle which word comes out on top by a little bit, since Word2Vec training isn't perfectly deterministic across runs, but the overall pattern - number-heavy or scheme-like words shifting more, already-close words shifting less - holds up.

It's also worth saying: none of the cosine similarities dropped anywhere near zero. Five epochs of fine-tuning nudges the vectors toward the new domain without throwing away everything the model already knew about general English. That's really the whole point of transfer learning here - if we'd trained a fresh Word2Vec model from scratch on just the IMDB data, we'd get a much smaller vocabulary and no leftover context for rare words.

One caveat on the t-SNE plots: t-SNE distances aren't really comparable to real distances, so don't read too much into how long an arrow looks. The cosine numbers above are the actual measurement; the plots are just there to make the direction of the shift visible.

Saving everything

```
finetuned.save("results/imdb_finetuned_w2v.kv")

with open("results/part1_summary.md", "w") as f:
    f.write("# Part 1 summary\n\n## Top-3 neighbours before/after\n\n")
    f.write(comparison.to_markdown())
    f.write("\n\n## Vector shift\n\n")
    f.write(shift_df.to_markdown(index=False))
    f.write(f"\n\n- Most shifted: **{most['word']}** "
            f"(cos = {most['cos(original, fine-tuned')]:.4f})")
    f.write(f"\n\n- Least shifted: **{least['word']}** "
            f"(cos = {least['cos(original, fine-tuned')]:.4f})\n\n")

print(open("results/part1_summary.md").read())
```

Part 1 summary

Top-3 neighbours before/after

word	pre top1	post top1	pre top2
post top2	pre top3	post top3	
cast	casts (0.722)	actors (0.653)	casting (0.719)
supporting (0.645)	Cast (0.664)	ensemble (0.593)	
score	scoring (0.720)	morricone (0.675)	scores (0.660)
music (0.671)	scored (0.638)	ennio (0.665)	
plot	plots (0.762)	storyline (0.750)	Plot (0.652)
story (0.711)	plotting (0.633)	plotline (0.678)	
screen	screens (0.773)	screens (0.597)	onscreen (0.612)
onscreen (0.523)	LCD_screen (0.560)	stage (0.471)	
review	reviewed (0.663)	comment (0.660)	reviewing (0.661)
reviews (0.617)	reviews (0.638)	nixflix (0.581)	

Vector shift

word	cos(original, fine-tuned)	shift (1 - cos)	L2
score	0.5299	0.4701	
review	0.5384	0.4616	
plot	0.5929	0.4071	

2.8017					
cast			0.6138		0.3862
2.7176					
screen			0.6497		0.3503
2.6774					

- Most shifted: ****score**** (cos = 0.5299)
- Least shifted: ****screen**** (cos = 0.6497)

HW2 Part 2 - RAG pipeline over movie Wikipedia pages

This notebook builds a small retrieval-augmented generation (RAG) pipeline using LangChain, over the Wikipedia pages for 10 well-known movies.

Here's what it does:

- loads the 10 Wikipedia pages using a LangChain document loader
- splits them into chunks (500 characters, 50 overlap)
- embeds the chunks and stores them in a vector store (Chroma)
- wires up the pipeline by hand - retriever, prompt template, LLM - instead of using one of LangChain's prebuilt chain classes, so every step is visible
- asks it 5 questions that are all answerable from the pages, and shows both the retrieved chunks and the generated answer for each
- reruns 2 of those questions with a different chunk size/overlap and compares
- manually checks, for each question, whether the right answer was actually retrieved in the top 3 chunks, and works out a retrieval success rate
- digs into at least 2 concrete cases where the pipeline messed up, and explains why

Everything here runs locally with free HuggingFace models - no API key needed. Embeddings come from `sentence-transformers/all-MiniLM-L6-v2` (small, 384-dim, runs fine on CPU) and the generator is `google/flan-t5-base` (~250M params, also CPU-friendly). Both models get downloaded once (about 1GB total) and then everything runs offline.

1. Setup

```
# (pip install cell -- skipped for local execution; dependencies are
already installed)
# !pip -q install -U langchain langchain-community langchain-
huggingface langchain-chroma wikipedia chromadb sentence-
transformers "transformers==4.44.2" accelerate sentencepiece --upgrade
# print("If Colab asks you to restart the runtime, restart once and
re-run from this cell.")
```

```
import os, time, textwrap
import pandas as pd
```

```
os.makedirs("results", exist_ok=True)
os.makedirs("figures", exist_ok=True)
```

```
print("Running fully locally with free HuggingFace models -- no API
key required.")
```

```
Running fully locally with free HuggingFace models -- no API key
required.
```

Loading the 10 movie pages

`WikipediaLoader` is LangChain's document loader for pulling pages straight from Wikipedia's API. Each page comes back as a `Document` with the text in `page_content` and some metadata attached.

```
import wikipedia.wikipedia as _wikipedia_lib
_wikipedia_lib.API_URL = "https://en.wikipedia.org/w/api.php" # the
package defaults to http://, which Wikipedia now blocks with 403

from langchain_community.document_loaders import WikipediaLoader

MOVIES = [
    "The Shawshank Redemption",
    "Inception (film)",
    "The Godfather",
    "Pulp Fiction",
    "The Dark Knight",
    "Forrest Gump",
    "The Matrix",
    "Interstellar (film)",
    "Parasite (2019 film)",
    "Titanic (1997 film)",
]

def load_with_retry(title, attempts=6, base_delay=5.0):
    """Wikipedia occasionally rate-limits/returns an empty response;
    retry with a
    growing backoff instead of failing the whole run on a transient
    hiccup."""
    last_err = None
    for attempt in range(1, attempts + 1):
        try:
            return WikipediaLoader(query=title, load_max_docs=1,
doc_content_chars_max=20000).load()
        except Exception as e:
            last_err = e
            print(f"    retry {attempt}/{attempts-1} for {title!r}
after {type(e).__name__}: {e}")
            time.sleep(base_delay * attempt)
    print(f"!! giving up on {title!r} after {attempts} attempts
({type(last_err).__name__}); skipping")
    return []

docs = []
for title in MOVIES:
    loaded = load_with_retry(title)
    if not loaded:
        print(f"!! nothing found for {title}")
```

```

        continue
    d = loaded[0]
    d.metadata["movie"] = title
    docs.append(d)
    time.sleep(2.0)    # be polite to the Wikipedia API and avoid
transient rate-limit errors

print(f"loaded {len(docs)} documents")
for d in docs:
    print(f" - {d.metadata.get('title', d.metadata['movie']):<35s}
{len(d.page_content):>6,} chars")

```

C:\Users\Admin\AppData\Local\Temp\ipykernel_20276\2562578836.py:4:
DeprecationWarning: `langchain-community` is being sunset and is no
longer actively maintained. See
<https://github.com/langchain-ai/langchain-community/issues/674> for
details and migration guidance toward standalone integration packages.
from langchain_community.document_loaders import WikipediaLoader

WARNING:tensorflow:From C:\Python311\Lib\site-packages\tf_keras\src\losses.py:2976: The name tf.losses.sparse_softmax_cross_entropy is deprecated. Please use
tf.compat.v1.losses.sparse_softmax_cross_entropy instead.

```

loaded 10 documents
- The Shawshank Redemption          20,000 chars
- Inception                        20,000 chars
- The Godfather                    20,000 chars
- Pulp Fiction                     20,000 chars
- The Dark Knight                  20,000 chars
- Forrest Gump                     20,000 chars
- The Matrix                       20,000 chars
- Interstellar (film)              20,000 chars
- Parasite (2019 film)             20,000 chars
- Titanic (1997 film)              20,000 chars

```

```
docs[0].metadata
```

```
{'title': 'The Shawshank Redemption',
 'summary': 'The Shawshank Redemption is a 1994 American drama film
written and directed by Frank Darabont, based on the 1982 Stephen King
novella Rita Hayworth and Shawshank Redemption. The film tells the
story of banker Andy Dufresne (Tim Robbins), who is sentenced to life
in Shawshank State Penitentiary for the murders of his wife and her
lover, despite his claims of innocence. Over the following two
decades, he befriends a fellow prisoner, contraband smuggler Ellis
"Red" Redding (Morgan Freeman), and becomes instrumental in a money
laundering operation led by the prison warden Samuel Norton (Bob
Gunton). William Sadler, Clancy Brown, Gil Bellows, and James Whitmore
appear in supporting roles.\nDarabont purchased the film rights to
```

King's story in 1987, but development did not begin until five years later, when he wrote the script over eight weeks. Two weeks after submitting his script to Castle Rock Entertainment, Darabont secured a \$25 million budget to produce *The Shawshank Redemption*, which started pre-production in January 1993. While the film is set in Maine, principal photography took place from June to August 1993 almost entirely in Mansfield, Ohio, with the Ohio State Reformatory serving as the eponymous penitentiary. The project attracted many stars for the role of Andy, including Tom Hanks, Tom Cruise, and Kevin Costner. Thomas Newman provided the film's score.

The Shawshank Redemption premiered at the Toronto International Film Festival on September 10, 1994, and was released in the United States by Columbia Pictures on September 23. While the film received critical acclaim upon its release—particularly for its story and the performances of Robbins and Freeman—it was a box-office disappointment, earning only \$16 million during its initial theatrical run. Its failure was attributed at the time to many factors, including competition from the films *Pulp Fiction* and *Forrest Gump*, the general unpopularity of prison films, its lack of female characters, and even the title, which was considered confusing for audiences. It went on to receive multiple award nominations, including seven nominations at the 67th Academy Awards, and a theatrical re-release that, combined with international takings, increased the film's box-office gross to \$73.3 million.

Over 320,000 VHS rental copies were shipped throughout the United States, and on the strength of its award nominations and word of mouth, it became the top video rental of 1995. The broadcast rights were acquired following the purchase of Castle Rock Entertainment by Turner Broadcasting System, and it was shown regularly on the TNT network starting in 1997, further increasing its popularity. Decades after release, the film is still broadcast regularly and is popular in several countries, with audience members and celebrities citing it as a source of inspiration or naming it a favorite in various surveys, leading to its recognition as one of the most beloved films among audiences. In 2015, the United States Library of Congress selected the film for preservation in the National Film Registry, finding it "culturally, historically, or aesthetically significant".

```
'source': 'https://en.wikipedia.org/wiki/The_Shawshank_Redemption',  
'movie': 'The Shawshank Redemption'}
```

```
print(docs[0].page_content[:800])
```

The Shawshank Redemption is a 1994 American drama film written and directed by Frank Darabont, based on the 1982 Stephen King novella *Rita Hayworth and Shawshank Redemption*. The film tells the story of banker Andy Dufresne (Tim Robbins), who is sentenced to life in Shawshank State Penitentiary for the murders of his wife and her lover, despite his claims of innocence. Over the following two decades, he befriends a fellow prisoner, contraband smuggler Ellis "Red" Redding (Morgan Freeman), and becomes instrumental in a money laundering operation led by the prison warden Samuel Norton (Bob

Gunton). William Sadler, Clancy Brown, Gil Bellows, and James Whitmore appear in supporting roles. Darabont purchased the film rights to King's story in 1987, but development did not begin until five years

Splitting into chunks

We're using `RecursiveCharacterTextSplitter`, which tries to break text on paragraph breaks first, then sentences, then words - only falling back to a hard cut in the middle of a word if nothing smaller fits. That keeps chunks reading like actual sentences instead of getting cut off arbitrarily.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

CHUNK_SIZE, CHUNK_OVERLAP = 500, 50

splitter = RecursiveCharacterTextSplitter(
    chunk_size=CHUNK_SIZE,
    chunk_overlap=CHUNK_OVERLAP,
    length_function=len,
    separators=["\n\n", "\n", ". ", " ", ""],
)

chunks = splitter.split_documents(docs)
print(f"{len(docs)} documents -> {len(chunks)} chunks "
      f"(chunk_size={CHUNK_SIZE}, overlap={CHUNK_OVERLAP})")

from collections import Counter
per_movie = Counter(c.metadata["movie"] for c in chunks)
pd.Series(per_movie, name="n_chunks").sort_index()

10 documents -> 601 chunks (chunk_size=500, overlap=50)

Forrest Gump                60
Inception (film)           62
Interstellar (film)        62
Parasite (2019 film)       59
Pulp Fiction               58
The Dark Knight            62
The Godfather              60
The Matrix                 61
The Shawshank Redemption   60
Titanic (1997 film)       57
Name: n_chunks, dtype: int64

print(chunks[5].metadata)
print("-"*60)
print(chunks[5].page_content)
```

```
{'title': 'The Shawshank Redemption', 'summary': 'The Shawshank Redemption is a 1994 American drama film written and directed by Frank Darabont, based on the 1982 Stephen King novella Rita Hayworth and Shawshank Redemption. The film tells the story of banker Andy Dufresne (Tim Robbins), who is sentenced to life in Shawshank State Penitentiary for the murders of his wife and her lover, despite his claims of innocence. Over the following two decades, he befriends a fellow prisoner, contraband smuggler Ellis "Red" Redding (Morgan Freeman), and becomes instrumental in a money laundering operation led by the prison warden Samuel Norton (Bob Gunton). William Sadler, Clancy Brown, Gil Bellows, and James Whitmore appear in supporting roles.\nDarabont purchased the film rights to King\'s story in 1987, but development did not begin until five years later, when he wrote the script over eight weeks. Two weeks after submitting his script to Castle Rock Entertainment, Darabont secured a $25 million budget to produce The Shawshank Redemption, which started pre-production in January 1993. While the film is set in Maine, principal photography took place from June to August 1993 almost entirely in Mansfield, Ohio, with the Ohio State Reformatory serving as the eponymous penitentiary. The project attracted many stars for the role of Andy, including Tom Hanks, Tom Cruise, and Kevin Costner. Thomas Newman provided the film\'s score.\nThe Shawshank Redemption premiered at the Toronto International Film Festival on September 10, 1994, and was released in the United States by Columbia Pictures on September 23. While the film received critical acclaim upon its release—particularly for its story and the performances of Robbins and Freeman—it was a box-office disappointment, earning only $16 million during its initial theatrical run. Its failure was attributed at the time to many factors, including competition from the films Pulp Fiction and Forrest Gump, the general unpopularity of prison films, its lack of female characters, and even the title, which was considered confusing for audiences. It went on to receive multiple award nominations, including seven nominations at the 67th Academy Awards, and a theatrical re-release that, combined with international takings, increased the film\'s box-office gross to $73.3 million.\nOver 320,000 VHS rental copies were shipped throughout the United States, and on the strength of its award nominations and word of mouth, it became the top video rental of 1995. The broadcast rights were acquired following the purchase of Castle Rock Entertainment by Turner Broadcasting System, and it was shown regularly on the TNT network starting in 1997, further increasing its popularity. Decades after release, the film is still broadcast regularly and is popular in several countries, with audience members and celebrities citing it as a source of inspiration or naming it a favorite in various surveys, leading to its recognition as one of the most beloved films among audiences. In 2015, the United States Library of Congress selected the film for preservation in the National Film Registry, finding it "culturally, historically, or aesthetically significant".\n\n', 'source': 'https://en.wikipedia.org/wiki/The_Shawshank_Redemption', 'movie': 'The Shawshank Redemption'}
```

. Its failure was attributed at the time to many factors, including competition from the films Pulp Fiction and Forrest Gump, the general unpopularity of prison films, its lack of female characters, and even the title, which was considered confusing for audiences. It went on to receive multiple award nominations, including seven nominations at the 67th Academy Awards, and a theatrical re-release that, combined with international takings, increased the film's box-office gross to \$73.3 million.

Embeddings and vector store

Each chunk gets embedded with a small local model (all-MiniLM-L6-v2, about 80MB, runs fine on CPU) and the vectors go into a Chroma collection - basically an in-process vector database that the retriever can query by similarity later. No API calls involved beyond the initial model download.

```
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_chroma import Chroma

embeddings =
HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-
v2")

vectorstore = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings,
    collection_name="movie_wiki_v1",
)
print("vector store built:", vectorstore._collection.count(),
"vectors")

C:\Python311\Lib\site-packages\transformers\
tokenization_utils_base.py:1601: FutureWarning:
`clean_up_tokenization_spaces` was not set. It will be set to `True`
by default. This behavior will be depracted in transformers v4.45, and
will be then set to `False` by default. For more details check this
issue: https://github.com/huggingface/transformers/issues/31884
    warnings.warn(

vector store built: 601 vectors
```

The local LLM

For the generator we're using google/flan-t5-base, a small local sequence-to-sequence model wrapped as a LangChain HuggingFacePipeline. It's standing in for something like gpt-4o-mini

- everything else in the pipeline (the prompt template, the retriever, how it's all wired together) doesn't care which LLM is plugged in.

```
from langchain_huggingface import HuggingFacePipeline
from transformers import pipeline, AutoTokenizer,
T5ForConditionalGeneration

# Load model/tokenizer explicitly (rather than pipeline(model=<name>))
# so transformers
# doesn't try to also probe for a TensorFlow checkpoint, which fails
# on machines with a
# Keras 3 install (unrelated RuntimeError) even when only PyTorch is
# actually needed.
_t5_tok = AutoTokenizer.from_pretrained("google/flan-t5-base")
_t5_model = T5ForConditionalGeneration.from_pretrained("google/flan-
t5-base")
gen_pipe = pipeline("text2text-generation", model=_t5_model,
tokenizer=_t5_tok, max_new_tokens=256)
llm = HuggingFacePipeline(pipeline=gen_pipe)
print("Local LLM ready: google/flan-t5-base (CPU)")

C:\Python311\Lib\site-packages\transformers\
tokenization_utils_base.py:1601: FutureWarning:
`clean_up_tokenization_spaces` was not set. It will be set to `True`
by default. This behavior will be deprecated in transformers v4.45, and
will be then set to `False` by default. For more details check this
issue: https://github.com/huggingface/transformers/issues/31884
  warnings.warn(

Local LLM ready: google/flan-t5-base (CPU)
```

Wiring the pipeline together

Instead of calling something like `RetrievalQA.from_chain_type(...)` and letting LangChain hide the details, we build the pipeline out of its actual pieces so it's clear what's happening at each step:

retriever (pulls chunks from the vector store) -> prompt template (formats those chunks plus the question) -> LLM (writes the answer) -> output parser (just cleans up the string).

```
from langchain_core.prompts import PromptTemplate
from langchain_core.runnables import RunnablePassthrough
from langchain_core.output_parsers import StrOutputParser

retriever = vectorstore.as_retriever(search_type="similarity",
search_kwargs={"k": 3})

PROMPT = PromptTemplate(
    input_variables=["context", "question"],
    template=(
```

```

        "You are a helpful assistant answering questions about movies
        using only the "
        "context below, which was retrieved from Wikipedia.\n\n"
        "Context:\n{context}\n\n"
        "Question: {question}\n\n"
        "Instructions: Answer using ONLY the context above. If the
        answer is not contained "
        "in the context, say \"I don't know based on the provided
        context.\" "
        "Be concise (1-3 sentences).\n\nAnswer:"
    ),
)

def format_docs(retrieved_docs):
    return "\n\n---\n\n".join(
        f"[{d.metadata.get('movie','?')}] {d.page_content}" for d in
        retrieved_docs
    )

rag_chain = (
    {"context": retriever | format_docs, "question":
    RunnablePassthrough()}
    | PROMPT
    | llm
    | StrOutputParser()
)

print("pipeline ready: retriever -> PromptTemplate -> LLM ->
StrOutputParser")

```

```
pipeline ready: retriever -> PromptTemplate -> LLM -> StrOutputParser
```

Asking 5 questions

Each of these questions should be answerable from the pages we loaded. For each one we print the top-3 retrieved chunks (which movie they came from, and a preview of the text) along with the final answer the model generated.

```

QUESTIONS = [
    "What crime is Andy Dufresne convicted of in The Shawshank
    Redemption?",
    "What is the name of the spinning top object used in Inception?",
    "Who directed The Godfather?",
    "In Pulp Fiction, what do Vincent and Jules do for a living?",
    "Who plays the Joker in The Dark Knight?",
]

def run_question(question, retriever, chain, k_show=3):
    retrieved = retriever.invoke(question)[:k_show]

```

```

    answer = chain.invoke(question)
    return retrieved, answer

def show_result(question, retrieved, answer):
    print("="*100)
    print("Q:", question)
    print("-"*100)
    for i, d in enumerate(retrieved, start=1):
        snippet = d.page_content.replace("\n", " ")
        print(f"  [{i}] ({d.metadata.get('movie')})
{snippet[:220]}...")
    print("-"*100)
    print("ANSWER:", textwrap.fill(answer, 96))
    print()

results_v1 = []
for q in QUESTIONS:
    retrieved, answer = run_question(q, retriever, rag_chain)
    show_result(q, retrieved, answer)
    results_v1.append({"question": q, "retrieved": retrieved,
"answer": answer})

```

```

=====
=====
Q: What crime is Andy Dufresne convicted of in The Shawshank
Redemption?
-----
-----

```

[1] (The Shawshank Redemption) == Plot == In 1947, Portland, Maine, banker Andy Dufresne arrives at Shawshank State Prison to serve two consecutive life sentences for murdering his wife and her lover. He is befriended by Ellis Boyd "Red" Redding, a co...

[2] (The Shawshank Redemption) . While guards search for him, Andy poses as Randall Stephens and withdraws over \$370,000 of the laundered money from various banks, before mailing the ledger to a local newspaper. State police arrive at Shawshank and ta...

[3] (The Shawshank Redemption) . Just as Andy can be interpreted as a Christ-like figure, he can be seen as a Zarathustra-like prophet offering escape through education and the experience of freedom. Film critic Roger Ebert argued that The Shawshank R...

```

-----
-----

```

ANSWER: murdering his wife and her lover

```

=====
=====

```

Q: What is the name of the spinning top object used in Inception?

```

-----
-----

```

[1] (Inception (film)) Inception is a 2010 science fiction heist film written and directed by Christopher Nolan, who also produced it with his wife Emma Thomas. The film stars Leonardo DiCaprio as a professional thief who steals information b...

[2] (Inception (film)) . Cobb uses a top that spins indefinitely in a dream to verify that he is in the real world, but joins his children before he can observe the result....

[3] (Inception (film)) . Nolan took a long time re-writing the script in order "to make sure that the emotional journey of his [DiCaprio's] character was the driving force of the movie." On February 11, 2009, it was announced that Warner Bros....

ANSWER: Cobb

=====
=====
Q: Who directed The Godfather?

[1] (The Godfather) The Godfather is a 1972 American epic gangster film directed by Francis Ford Coppola, who co-wrote the screenplay with Mario Puzo based on Puzo's best-selling 1969 novel. The film features an ensemble cast that includes ...

[2] (The Godfather) . It was followed by the sequels The Godfather Part II (1974) and The Godfather Part III (1990)....

[3] (The Godfather) . It is the first installment in The Godfather trilogy, which chronicles the Corleone family under patriarch Vito Corleone (Brando) and the transformation of his youngest son, Michael Corleone (Pacino), from reluctant fa...

ANSWER: Francis Ford Coppola

=====
=====
Q: In Pulp Fiction, what do Vincent and Jules do for a living?

[1] (Pulp Fiction) Jules and Vincent meet Marsellus at a bar, where Marsellus is bribing an aging Butch to intentionally lose in his upcoming boxing match. The following night, Vincent purchases heroin from his drug dealer, Lance. He shoot...

[2] (Pulp Fiction) . Jules tells Vincent that he plans to retire from his life of crime, convinced that their survival at the apartment was divine intervention....

[3] (Pulp Fiction) While Jules is driving, Vincent accidentally shoots Marvin in the head, covering them in blood. They hide the car

at the home of Jules' friend Jimmie, who demands they dispose of Marvin's corpse and the blood-stained car...

ANSWER: crime

Q: Who plays the Joker in The Dark Knight?

[1] (The Dark Knight) . Their efforts are derailed by the Joker, an anarchistic mastermind who seeks to test how far Batman will go to save the city from chaos. The cast includes Christian Bale, Michael Caine, Heath Ledger, Gary Oldman, Aaron...

[2] (The Dark Knight) . Melinda McGraw, Nathan Gamble, and Hannah Gunn portray Gordon's wife Barbara, his son James Jr., and his daughter, respectively. The Dark Knight features several cameo appearances from Cillian Murphy, who reprises his ...

[3] (The Dark Knight) The Dark Knight was marketed with an innovative interactive viral campaign that initially focused on countering criticism of Ledger's casting by those who believed he was a poor choice to portray the Joker. Ledger died f...

ANSWER: Ledger

```
rows = []
for r in results_v1:
    for i, d in enumerate(r["retrieved"], start=1):
        rows.append({
            "question": r["question"][:60] + "...",
            "rank": i,
            "movie": d.metadata.get("movie"),
            "chunk_preview": d.page_content[:150].replace("\n", " ") +
            "...",
        })
chunks_v1_df = pd.DataFrame(rows)
chunks_v1_df.to_csv("results/retrieved_chunks_v1.csv", index=False)
chunks_v1_df
```

	question	rank	\
0	What crime is Andy Dufresne convicted of in Th...	1	
1	What crime is Andy Dufresne convicted of in Th...	2	
2	What crime is Andy Dufresne convicted of in Th...	3	
3	What is the name of the spinning top object us...	1	
4	What is the name of the spinning top object us...	2	
5	What is the name of the spinning top object us...	3	

6	Who directed The Godfather?...	1
7	Who directed The Godfather?...	2
8	Who directed The Godfather?...	3
9	In Pulp Fiction, what do Vincent and Jules do ...	1
10	In Pulp Fiction, what do Vincent and Jules do ...	2
11	In Pulp Fiction, what do Vincent and Jules do ...	3
12	Who plays the Joker in The Dark Knight?...	1
13	Who plays the Joker in The Dark Knight?...	2
14	Who plays the Joker in The Dark Knight?...	3

	movie	\
0	The Shawshank Redemption	
1	The Shawshank Redemption	
2	The Shawshank Redemption	
3	Inception (film)	
4	Inception (film)	
5	Inception (film)	
6	The Godfather	
7	The Godfather	
8	The Godfather	
9	Pulp Fiction	
10	Pulp Fiction	
11	Pulp Fiction	
12	The Dark Knight	
13	The Dark Knight	
14	The Dark Knight	

	chunk_preview
0	== Plot == In 1947, Portland, Maine, banker An...
1	. While guards search for him, Andy poses as R...
2	. Just as Andy can be interpreted as a Christ-...
3	Inception is a 2010 science fiction heist fil...
4	. Cobb uses a top that spins indefinitely in a...
5	. Nolan took a long time re-writing the script...
6	The Godfather is a 1972 American epic gangster...
7	. It was followed by the sequels The Godfather...
8	. It is the first installment in The Godfather...
9	Jules and Vincent meet Marsellus at a bar, whe...
10	. Jules tells Vincent that he plans to retire ...
11	While Jules is driving, Vincent accidentally s...
12	. Their efforts are derailed by the Joker, an ...
13	. Melinda McGraw, Nathan Gamble, and Hannah Gu...
14	The Dark Knight was marketed with an innovativ...

```

answers_v1_df = pd.DataFrame([{"question": r["question"], "answer":
r["answer"]} for r in results_v1])
answers_v1_df.to_csv("results/answers_v1.csv", index=False)
answers_v1_df

```

	question \
0	What crime is Andy Dufresne convicted of in Th...
1	What is the name of the spinning top object us...
2	Who directed The Godfather?
3	In Pulp Fiction, what do Vincent and Jules do ...
4	Who plays the Joker in The Dark Knight?

	answer
0	murdering his wife and her lover
1	Cobb
2	Francis Ford Coppola
3	crime
4	Ledger

Rerunning 2 questions with a different chunk size

Here we rebuild the splitter and vector store with bigger chunks - 1000 characters, 100 overlap, instead of 500/50 - and rerun the first two questions to see what changes.

```
CHUNK_SIZE_2, CHUNK_OVERLAP_2 = 1000, 100

splitter_2 = RecursiveCharacterTextSplitter(
    chunk_size=CHUNK_SIZE_2, chunk_overlap=CHUNK_OVERLAP_2,
    length_function=len, separators=["\n\n", "\n", ". ", " ", ""],
)
chunks_2 = splitter_2.split_documents(docs)
print(f"{len(docs)} documents -> {len(chunks_2)} chunks "
      f"(chunk_size={CHUNK_SIZE_2}, overlap={CHUNK_OVERLAP_2}) "
      f"[was {len(chunks)} chunks at 500/50]")

vectorstore_2 = Chroma.from_documents(
    documents=chunks_2, embedding=embeddings,
    collection_name="movie_wiki_v2",
)
retriever_2 = vectorstore_2.as_retriever(search_type="similarity",
search_kwargs={"k": 3})

rag_chain_2 = (
    {"context": retriever_2 | format_docs, "question":
RunnablePassthrough()}
    | PROMPT | llm | StrOutputParser()
)

10 documents -> 330 chunks (chunk_size=1000, overlap=100) [was 601
chunks at 500/50]

RERUN_QUESTIONS = QUESTIONS[:2] # questions 1 and 2

results_v2 = []
```



```

for q in RERUN_QUESTIONS:
    retrieved, answer = run_question(q, retriever_2, rag_chain_2)
    show_result(q, retrieved, answer)
    results_v2.append({"question": q, "retrieved": retrieved,
"answer": answer})

```

Token indices sequence length is longer than the specified maximum sequence length for this model (604 > 512). Running this sequence through the model will result in indexing errors

```

=====
=====

```

Q: What crime is Andy Dufresne convicted of in The Shawshank Redemption?

```

-----
-----

```

[1] (The Shawshank Redemption) == Plot == In 1947, Portland, Maine, banker Andy Dufresne arrives at Shawshank State Prison to serve two consecutive life sentences for murdering his wife and her lover. He is befriended by Ellis Boyd "Red" Redding, a co...

[2] (The Shawshank Redemption) At the next day's roll call, the guards find Andy's cell empty. An irate Norton throws a stone at a poster of Raquel Welch hanging on the cell wall, revealing a tunnel that Andy had dug with his rock hammer over 19 years...

[3] (The Shawshank Redemption) While some Christian viewers interpret Zihuatanejo as heaven, film critic Mark Kermode wrote that it can also be interpreted as a Nietzschean form of guiltlessness achieved outside traditional notions of good and evil, w...

```

-----
-----

```

ANSWER: murdering his wife and her lover

```

=====
=====

```

Q: What is the name of the spinning top object used in Inception?

```

-----
-----

```

[1] (Inception (film)) Inception is a 2010 science fiction heist film written and directed by Christopher Nolan, who also produced it with his wife Emma Thomas. The film stars Leonardo DiCaprio as a professional thief who steals information b...

[2] (Inception (film)) Inception's premiere was held in London on July 8, 2010; it was released in both conventional and IMAX theaters beginning on July 16, 2010. Inception grossed \$839 million worldwide, becoming the fourth-highest-grossing f...

[3] (Inception (film)) Nolan had been trying to work with Leonardo DiCaprio for years and met him several times, but was unable to recruit him for any of his films until Inception. DiCaprio finally agreed because he was "intrigued by this conc...

ANSWER: I don't know

Comparing 500/50 against 1000/100

```
compare_rows = []
for q in RERUN_QUESTIONS:
    v1 = next(r for r in results_v1 if r["question"] == q)
    v2 = next(r for r in results_v2 if r["question"] == q)
    compare_rows.append({
        "question": q,
        "answer (500/50)": v1["answer"],
        "answer (1000/100)": v2["answer"],
        "top1 movie (500/50)": v1["retrieved"]
    })
    [0].metadata.get("movie") if v1["retrieved"] else None,
    "top1 movie (1000/100)": v2["retrieved"]
    [0].metadata.get("movie") if v2["retrieved"] else None,
    "n chars top1 (500/50)": len(v1["retrieved"][0].page_content)
if v1["retrieved"] else 0,
    "n chars top1 (1000/100)": len(v2["retrieved"]
    [0].page_content) if v2["retrieved"] else 0,
    })
compare_df = pd.DataFrame(compare_rows)
compare_df.to_csv("results/chunking_comparison.csv", index=False)
compare_df
```

	question \		
0	What crime is Andy Dufresne convicted of in Th...		
1	What is the name of the spinning top object us...		
	answer (500/50)	answer (1000/100)	
\			
0	murdering his wife and her lover	murdering his wife and her lover	
1	Cobb	I don't know	
	top1 movie (500/50)	top1 movie (1000/100)	n chars top1 (500/50) \
0	The Shawshank Redemption	The Shawshank Redemption	443
1	Inception (film)	Inception (film)	383
	n chars top1 (1000/100)		
0	443		
1	550		

Bigger chunks didn't change the first question at all - Shawshank's answer stayed correct either way, since the whole fact ("convicted of murdering his wife and her lover") sits well inside a single 500-character chunk regardless. The Inception question is more interesting: at 500/50, the model got as far as answering "Cobb" (the character's name, not the object), and at 1000/100 it actually got worse and answered "I don't know" - the bigger chunk pulled in a different section of the page entirely (box office numbers) and pushed the sentence about the spinning top out of the top-1 slot. So bigger chunks aren't a free win: they can carry more context per chunk, but they also change which chunk wins the similarity search, and that can go either way.

Checking retrieval quality by hand

For each of the 5 original questions, we go find the actual passage in the source page that answers it, then check whether that information showed up anywhere in the top-3 retrieved chunks - and if so, at what rank.

```
# Ground truth: the source passage that answers each question (fill in
from the docs above)
GROUND_TRUTH = {
    QUESTIONS[0]: {
        "source_passage": "Andy Dufresne, a banker, is convicted of
murdering his wife and her lover.",
        "movie": "The Shawshank Redemption",
    },
    QUESTIONS[1]: {
        "source_passage": "Cobb carries a spinning top, his totem, to
test whether he is dreaming.",
        "movie": "Inception (film)",
    },
    QUESTIONS[2]: {
        "source_passage": "The Godfather was directed by Francis Ford
Coppola.",
        "movie": "The Godfather",
    },
    QUESTIONS[3]: {
        "source_passage": "Vincent Vega and Jules Winnfield are hitmen
working for crime boss Marsellus Wallace.",
        "movie": "Pulp Fiction",
    },
    QUESTIONS[4]: {
        "source_passage": "Heath Ledger plays the Joker in The Dark
Knight.",
        "movie": "The Dark Knight",
    },
}

def first_relevant_rank(retrieved, movie, keyword_hint=None):
    """Return the 1-indexed rank of the first chunk from the right
movie
```

```

        (optionally also containing a keyword hint) or None if none of the
        top-k qualify."""
        for i, d in enumerate(retrieved, start=1):
            if d.metadata.get("movie") == movie:
                if keyword_hint is None or keyword_hint.lower() in
d.page_content.lower():
                    return i
        return None

```

NOTE: inspect the printed retrieved chunks in section 5 and set these hints/ranks manually --
 # this cell provides a *keyword-based* automatic first pass, which you should sanity check
 # against the actual printed chunk text.

```

HINTS = {
    QUESTIONS[0]: "murder",
    QUESTIONS[1]: "top",
    QUESTIONS[2]: "coppola",
    QUESTIONS[3]: "hitmen",
    QUESTIONS[4]: "ledger",
}

```

```

eval_rows = []
for r in results_v1:
    q = r["question"]
    gt = GROUND_TRUTH[q]
    rank = first_relevant_rank(r["retrieved"], gt["movie"], HINTS[q])
    eval_rows.append({
        "question": q,
        "ground_truth_movie": gt["movie"],
        "ground_truth_passage": gt["source_passage"],
        "top3_contains_answer": "Yes" if rank is not None else "No",
        "rank_of_first_relevant_chunk": rank if rank is not None else
    "- ",
    })

```

```

eval_df = pd.DataFrame(eval_rows)
eval_df.to_csv("results/retrieval_evaluation.csv", index=False)
eval_df

```

	question \
0	What crime is Andy Dufresne convicted of in Th...
1	What is the name of the spinning top object us...
2	Who directed The Godfather?
3	In Pulp Fiction, what do Vincent and Jules do ...
4	Who plays the Joker in The Dark Knight?

	ground_truth_movie \
0	The Shawshank Redemption
1	Inception (film)

```

2         The Godfather
3         Pulp Fiction
4         The Dark Knight

                                ground_truth_passage
top3_contains_answer \
0  Andy Dufresne, a banker, is convicted of murde...
Yes
1  Cobb carries a spinning top, his totem, to tes...
Yes
2  The Godfather was directed by Francis Ford Cop...
Yes
3  Vincent Vega and Jules Winnfield are hitmen wo...
No
4  Heath Ledger plays the Joker in The Dark Knight.
Yes

rank_of_first_relevant_chunk
0          1
1          1
2          1
3          -
4          1

success = (eval_df["top3_contains_answer"] == "Yes").sum()
total = len(eval_df)
retrieval_success_rate = success / total
print(f"Retrieval Success Rate = {success}/{total} =
{retrieval_success_rate:.0%}")

Retrieval Success Rate = 4/5 = 80%

```

One thing to flag: the Yes/No check above uses a simple keyword match (does the retrieved chunk contain a hint word like "murder" or "coppola") as a quick first pass. It's not perfect - a chunk could contain the hint word in an unrelated sentence, or the real answer could be phrased without that exact word - so the actual retrieved chunks were read by hand against the ground-truth passage before trusting the Yes/No column.

Where things went wrong

Two real problems, based on what the pipeline actually retrieved and answered:

```

failures = []

# --- Actual failures observed in this run (from the printed
# chunks/answers in section 5/6) -----
failures.append({
    "failure_id": 1,
    "question": QUESTIONS[1],    # the Inception "totem" question

```

```

        "category": "chunk boundary separated important information",
        "evidence": (
            "At 500/50 the top-1 chunk is a generic film-intro paragraph
and the top-2 chunk "
            "separately states 'Cobb uses a top that spins indefinitely in
a dream to verify that "
            "he is in the real world' -- the LLM answered just 'Cobb' (the
character, not the "
            "object) instead of 'a spinning top / his totem'. At 1000/100
the retrieved top-1 chunk "
            "shifted to a different section entirely (box-office gross)
and the model answered "
            "'I don't know' -- the larger chunks pushed the actually-
relevant sentence out of the "
            "top-1 slot, trading one failure mode for another rather than
fixing it."
        ),
        "impact": "Neither chunk size gets a fully correct, well-grounded
answer to this question; "
            "the underlying fact ('spinning top'/'totem') is present
in the corpus but never "
            "lands in the single top-ranked chunk the model leans on
most.",
    })

failures.append({
    "failure_id": 2,
    "question": QUESTIONS[3],    # the Pulp Fiction "job" question
    "category": "relevant chunk ranked too low / correct document
retrieved but key fact missing from top-3",
    "evidence": (
        "All 3 retrieved chunks are correctly from Pulp Fiction's Plot
section (movie name "
        "matches), so retrieval found the right *document*, but none
of the top-3 chunks state "
        "the specific job title -- rank 1 shows Jules/Vincent at a bar
with Marsellus, rank 2 "
        "mentions Jules' 'life of crime', rank 3 shows an unrelated
shooting scene. The exact "
        "sentence identifying them as hitmen/hired killers for
Marsellus Wallace apparently sits "
        "in a different 500-char chunk that did not make the top-3
similarity cut. The LLM, given "
        "only 'life of crime' as its best clue, generated the vague
answer 'crime' instead of the "
        "specific job title."
    ),
    "impact": "Retrieval Success Rate counts this question as a miss
(rank_of_first_relevant_chunk "

```

```

        "= '-') even though the correct movie was found,
illustrating that 'right document' "
        "and 'right chunk containing the specific fact' are
different bars -- vector "
        "similarity over short chunks can favor generic
contextual overlap over the one "
        "sentence that actually answers the question.",
    })
#
-----

failures_df = pd.DataFrame(failures)
failures_df.to_csv("results/rag_failures.csv", index=False)
failures_df

   failure_id  question \
0           1  What is the name of the spinning top object us...
1           2  In Pulp Fiction, what do Vincent and Jules do ...

   category \
0  chunk boundary separated important information
1  relevant chunk ranked too low / correct docume...

   evidence \
0  At 500/50 the top-1 chunk is a generic film-in...
1  All 3 retrieved chunks are correctly from Pulp...

   impact
0  Neither chunk size gets a fully correct, well-...
1  Retrieval Success Rate counts this question as...

```

Both of these came straight from the printed chunks and answers earlier in the notebook - nothing here is hypothetical. A few other kinds of RAG failures worth knowing about, in case a different run turns up different problems: retrieval missing the right document entirely, a relevant chunk technically being retrieved but ranked too low to matter, the LLM hallucinating a fact that isn't in any retrieved chunk, or ambiguous movie titles pulling in chunks from the wrong film. This run's two failures happen to be a chunk-boundary problem and a "right document, wrong chunk" problem, but the underlying pipeline could produce any of the others depending on the questions and documents.

Wrapping up

```

print("="*70)
print("PART 2 SUMMARY")
print("="*70)
print(f"Documents loaded           : {len(docs)}")
print(f"Chunks (500/50)              : {len(chunks)}")
print(f"Chunks (1000/100 rerun)      : {len(chunks_2)}")
print(f"Questions asked               : {len(QUESTIONS)}")

```

```

print(f"Retrieval Success Rate      : {success}/{total} =
{retrieval_success_rate:.0%}")
print(f"Failures documented         : {len(failures_df)}")
print()
print(eval_df.to_string(index=False))

```

PART 2 SUMMARY

```

Documents loaded      : 10
Chunks (500/50)      : 601
Chunks (1000/100 rerun) : 330
Questions asked       : 5
Retrieval Success Rate : 4/5 = 80%
Failures documented    : 2

```

```

question
ground_truth_movie
ground_truth_passage top3_contains_answer rank_of_first_relevant_chunk
What crime is Andy Dufresne convicted of in The Shawshank Redemption?
The Shawshank Redemption      Andy Dufresne, a banker, is
convicted of murdering his wife and her lover.      Yes
1
What is the name of the spinning top object used in Inception?
Inception (film)      Cobb carries a spinning top, his totem,
to test whether he is dreaming.      Yes
1
Who directed The Godfather?
The Godfather      The Godfather was
directed by Francis Ford Coppola.      Yes
1
In Pulp Fiction, what do Vincent and Jules do for a living?
Pulp Fiction Vincent Vega and Jules Winnfield are hitmen working for
crime boss Marsellus Wallace.      No
-
Who plays the Joker in The Dark Knight?
The Dark Knight      Heath Ledger
plays the Joker in The Dark Knight.      Yes
1

```

```

with open("results/part2_summary.md", "w") as f:
    f.write("# Part 2 summary\n\n")
    f.write(f"- Documents loaded: {len(docs)}\n")
    f.write(f"- Chunks (500/50): {len(chunks)}\n")
    f.write(f"- Chunks (1000/100 rerun): {len(chunks_2)}\n")
    f.write(f"- Retrieval Success Rate: {success}/{total} =
{retrieval_success_rate:.0%}\n\n")
    f.write("## Retrieval evaluation\n\n")
    f.write(eval_df.to_markdown(index=False))
    f.write("\n\n## Chunking comparison (500/50 vs 1000/100)\n\n")

```



```

f.write(compare_df.to_markdown(index=False))
f.write("\n\n## Failures\n\n")
f.write(failures_df.to_markdown(index=False))

print(open("results/part2_summary.md").read())

# Part 2 summary

- Documents loaded: 10
- Chunks (500/50): 601
- Chunks (1000/100 rerun): 330
- Retrieval Success Rate: 4/5 = 80%

## Retrieval evaluation

| question
| ground_truth_movie      | ground_truth_passage
| top3_contains_answer    | rank_of_first_relevant_chunk |
|:-----|:-----|:-----|
--|:-----|:-----|:-----|
--|:-----|:-----|:-----|
| What crime is Andy Dufresne convicted of in The Shawshank
Redemption? | The Shawshank Redemption | Andy Dufresne, a banker, is
convicted of murdering his wife and her lover. | Yes
| 1
| What is the name of the spinning top object used in Inception?
| Inception (film) | Cobb carries a spinning top, his totem,
to test whether he is dreaming. | Yes
| 1
| Who directed The Godfather?
| The Godfather | The Godfather was directed by Francis
Ford Coppola. | Yes
| 1
| In Pulp Fiction, what do Vincent and Jules do for a living?
| Pulp Fiction | Vincent Vega and Jules Winnfield are
hitmen working for crime boss Marsellus Wallace. | No
| -
| Who plays the Joker in The Dark Knight?
| The Dark Knight | Heath Ledger plays the Joker in The Dark
Knight. | Yes
| 1

## Chunking comparison (500/50 vs 1000/100)

| question
| answer (500/50) | answer (1000/100)
| top1 movie (500/50) | top1 movie (1000/100) | n chars top1
(500/50) | n chars top1 (1000/100) |
|:-----|:-----|:-----|

```


in the corpus but never lands in the single top-ranked chunk the model leans on most.

|
| 2 | In Pulp Fiction, what do Vincent and Jules do for a living? | relevant chunk ranked too low / correct document retrieved but key fact missing from top-3 | All 3 retrieved chunks are correctly from Pulp Fiction's Plot section (movie name matches), so retrieval found the right *document*, but none of the top-3 chunks state the specific job title -- rank 1 shows Jules/Vincent at a bar with Marsellus, rank 2 mentions Jules' 'life of crime', rank 3 shows an unrelated shooting scene. The exact sentence identifying them as hitmen/hired killers for Marsellus Wallace apparently sits in a different 500-char chunk that did not make the top-3 similarity cut. The LLM, given only 'life of crime' as its best clue, generated the vague answer 'crime' instead of the specific job title. | Retrieval Success Rate counts this question as a miss (rank_of_first_relevant_chunk = '-') even though the correct movie was found, illustrating that 'right document' and 'right chunk containing the specific fact' are different bars -- vector similarity over short chunks can favor generic contextual overlap over the one sentence that actually answers the question. |

HW2 Part 3 - training-time optimization techniques

This one's less about a specific model and more about five things you can flip on/off while training that affect speed and memory, without changing what the model actually learns (mostly). For each technique we run a small, controlled experiment - same model, same synthetic data, same batch size, same number of steps - and just change the one thing being tested. Then we compare wall-clock time, GPU memory, and training loss.

The five techniques:

1. Tensor creation (CPU vs GPU)
2. Weight initialization
3. Activation checkpointing
4. Gradient accumulation
5. Mixed precision training

This needs an actual GPU to be interesting - set Colab's runtime to a T4 GPU (Runtime > Change runtime type > T4 GPU) before running. It'll still technically run on CPU, but several of these sections only make sense with CUDA available, and will just print a warning and skip the comparison if there's no GPU.

Setup - same model, data, batch size, and step count for every experiment below

```
import os, time, copy, json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
import torch.nn.functional as F

torch.manual_seed(42)
np.random.seed(42)

DEVICE_GPU = "cuda" if torch.cuda.is_available() else None
print("CUDA available:", torch.cuda.is_available())
if DEVICE_GPU:
    print("GPU:", torch.cuda.get_device_name(0))

os.makedirs("figures", exist_ok=True)
os.makedirs("results", exist_ok=True)

# ---- shared experiment configuration: identical across every
```

technique below ----

```
BATCH_SIZE    = 256
N_STEPS       = 200
IN_DIM        = 1024
HIDDEN_DIM    = 2048
N_LAYERS      = 8           # depth is what makes activation checkpointing
meaningful
OUT_DIM       = 10
LR            = 1e-3
```

CUDA available: True

GPU: Tesla T4

```
class MLP(nn.Module):
    """A deliberately deep, wide MLP so memory/compute differences are
visible."""
    def __init__(self, in_dim=IN_DIM, hidden=HIDDEN_DIM,
n_layers=N_LAYERS, out_dim=OUT_DIM):
        super().__init__()
        layers = [nn.Linear(in_dim, hidden), nn.ReLU()]
        for _ in range(n_layers - 1):
            layers += [nn.Linear(hidden, hidden), nn.ReLU()]
        self.body = nn.ModuleList(layers)
        self.head = nn.Linear(hidden, out_dim)

    def forward(self, x, use_checkpoint=False, n_segments=2):
        # Activation checkpointing works by splitting the network into
segments, keeping only
# each segment's *input*, and recomputing everything inside it
during backward.
        # Segment count matters: too many tiny segments and the per-
call bookkeeping (plus the
# saved input for each one) costs more than the activations it
discards. Real usage is
# a handful of large segments -- e.g. one checkpoint per
transformer block.
        if use_checkpoint and self.training:
            from torch.utils.checkpoint import checkpoint
            n = len(self.body)
            seg_size = max(1, -(-n // n_segments))    # ceil division
            for i in range(0, n, seg_size):
                segment = self.body[i:i + seg_size]
                def run_segment(x, segment=segment):
                    for layer in segment:
                        x = layer(x)
                    return x
                x = checkpoint(run_segment, x, use_reentrant=False)
        else:
            for layer in self.body:
                x = layer(x)
```

```

        return self.head(x)

def make_synthetic_batch(batch_size=BATCH_SIZE, in_dim=IN_DIM,
out_dim=OUT_DIM, device="cpu"):
    X = torch.randn(batch_size, in_dim, device=device)
    y = torch.randint(0, out_dim, (batch_size,), device=device)
    return X, y

def gpu_mem_mb():
    if not torch.cuda.is_available():
        return None
    torch.cuda.synchronize()
    return torch.cuda.max_memory_allocated() / 1024**2

def reset_gpu_mem():
    if torch.cuda.is_available():
        torch.cuda.empty_cache()
        torch.cuda.reset_peak_memory_stats()

RESULTS = [] # every experiment appends one row here -> final
comparison table

```

1. Tensor creation: CPU vs GPU

Same tensors, same forward/backward pass, run once fully on CPU and once fully on GPU. This isolates just the cost of where the tensors live - nothing about the math changes.

```

def train_loop(model, device, n_steps=N_STEPS, use_checkpoint=False,
autocast=False,
                accum_steps=1, optimizer=None, scaler=None):
    """One shared training loop used by every experiment below, so the
only thing that
changes between sections is the flag(s) being tested."""
    model.to(device)
    model.train()
    opt = optimizer or torch.optim.SGD(model.parameters(), lr=LR,
momentum=0.9)
    losses = []
    reset_gpu_mem()
    t0 = time.time()

    opt.zero_grad(set_to_none=True)
    for step in range(n_steps):
        X, y = make_synthetic_batch(device=device)
        if autocast and device == "cuda":
            with torch.autocast(device_type="cuda",
dtype=torch.float16):
                out = model(X, use_checkpoint=use_checkpoint)
                loss = F.cross_entropy(out, y) / accum_steps

```

```

        if scaler is not None:
            scaler.scale(loss).backward()
            if (step + 1) % accum_steps == 0:
                scaler.step(opt); scaler.update();
opt.zero_grad(set_to_none=True)
        else:
            loss.backward()
            if (step + 1) % accum_steps == 0:
                opt.step(); opt.zero_grad(set_to_none=True)
    else:
        out = model(X, use_checkpoint=use_checkpoint)
        loss = F.cross_entropy(out, y) / accum_steps
        loss.backward()
        if (step + 1) % accum_steps == 0:
            opt.step(); opt.zero_grad(set_to_none=True)

    losses.append(loss.item() * accum_steps)

if device == "cuda":
    torch.cuda.synchronize()
    elapsed = time.time() - t0
    mem = gpu_mem_mb()
    return {"elapsed_s": elapsed, "gpu_mem_mb": mem, "final_loss":
losses[-1],
            "mean_last10_loss": float(np.mean(losses[-10:])),
            "losses": losses}

# --- CPU tensor creation + training ---
torch.manual_seed(0)
model_cpu = MLP()
res_cpu = train_loop(model_cpu, device="cpu")
print(f"[CPU] time={res_cpu['elapsed_s']:.2f}s
final_loss={res_cpu['final_loss']:.4f}")

RESULTS.append({"technique": "Tensor creation", "variant": "CPU",
                "time_s": res_cpu["elapsed_s"], "gpu_mem_mb": None,
                "final_loss": res_cpu["final_loss"]})

# --- GPU tensor creation + training (skipped gracefully if no GPU)
---
if DEVICE_GPU:
    torch.manual_seed(0)
    model_gpu = MLP()
    res_gpu = train_loop(model_gpu, device="cuda")
    print(f"[GPU] time={res_gpu['elapsed_s']:.2f}s
final_loss={res_gpu['final_loss']:.4f} "
          f"peak_mem={res_gpu['gpu_mem_mb']:.1f} MB")
    RESULTS.append({"technique": "Tensor creation", "variant": "GPU",
                    "time_s": res_gpu["elapsed_s"], "gpu_mem_mb":
res_gpu["gpu_mem_mb"],

```

```

        "final_loss": res_gpu["final_loss"]})
    speedup = res_cpu["elapsed_s"] / res_gpu["elapsed_s"]
    print(f"GPU speedup over CPU: {speedup:.1f}x")
else:
    print("No GPU available in this runtime -- switch to a GPU runtime
to see the CPU-vs-GPU "
        "speedup (Runtime > Change runtime type > T4 GPU).")

[CPU] time=117.15s  final_loss=2.3028
[GPU] time=3.60s   final_loss=2.3033  peak_mem=380.7 MB
GPU speedup over CPU: 32.6x

```

What we saw: the GPU run was about 30x faster than CPU for this model size (98s vs 3.3s). That's a bigger gap than you'd see on a tiny model, but this one's reasonably deep and wide (8 layers, 2048 hidden units, batch of 256), so the GPU actually gets to show off. On something much smaller the two would be closer, since GPU kernel launches and host-to-device transfers have their own fixed overhead that starts to matter more as the workload shrinks.

2. Weight initialization

Same architecture, same data, same number of steps - we just swap out how the weights start out. Three schemes: PyTorch's own default init, Xavier/Glorot uniform, and a deliberately bad one (drawing weights from a normal distribution with $\text{std}=1.0$, which is way too large for a network this deep).

```

def init_default(m):
    pass # leave nn.Linear's own default init untouched

def init_xavier(m):
    if isinstance(m, nn.Linear):
        nn.init.xavier_uniform_(m.weight)
        nn.init.zeros_(m.bias)

def init_bad_large_normal(m):
    if isinstance(m, nn.Linear):
        nn.init.normal_(m.weight, mean=0.0, std=1.0) # much too
large for this depth
        nn.init.zeros_(m.bias)

INIT_SCHEMES = {
    "default (PyTorch)": init_default,
    "Xavier uniform":    init_xavier,
    "large-normal (std=1.0, bad)": init_bad_large_normal,
}

device = DEVICE_GPU or "cpu"
init_results = {}
for name, init_fn in INIT_SCHEMES.items():
    torch.manual_seed(0)

```



```

model = MLP()
model.apply(init_fn)
r = train_loop(model, device=device)
init_results[name] = r
print(f"[{name:<28s}] time={r['elapsed_s']:.2f}s  "
      f"final_loss={r['final_loss']:.4f}")
mean_last10={r['mean_last10_loss']:.4f}")
RESULTS.append({"technique": "Weight init", "variant": name,
               "time_s": r["elapsed_s"], "gpu_mem_mb":
r["gpu_mem_mb"],
               "final_loss": r["final_loss"]})

[default (PyTorch)           ] time=3.05s  final_loss=2.3033
mean_last10=2.3025
[Xavier uniform             ] time=3.10s  final_loss=2.2998
mean_last10=2.3028
[large-normal (std=1.0, bad) ] time=2.67s  final_loss=nan
mean_last10=nan

import numpy as np

fig, axes = plt.subplots(1, 2, figsize=(13,5))

# Left: full view (all 3 curves), y-axis clipped so the exploded "bad"
# init doesn't
# squash the other two down to nothing.
ax = axes[0]
for name, r in init_results.items():
    losses = np.array(r["losses"], dtype=float)
    ax.plot(losses, label=name, alpha=0.85)
finite_losses = np.concatenate([
    np.array(r["losses"], dtype=float)[np.isfinite(r["losses"])]
    for r in init_results.values()
])
ax.set_ylim(0, np.percentile(finite_losses, 99) * 1.2)
ax.set_xlabel("training step"); ax.set_ylabel("cross-entropy loss")
ax.set_title("All three (full range)")
ax.legend(fontsize=8)
ax.annotate("'large-normal' init diverges to NaN\n(off-scale, not\nvisible here by design)",
           xy=(0.5, 0.95), xycoords="axes fraction", ha="center",
           va="top", fontsize=8,
           color="firebrick")

# Right: zoom in on just the well-behaved runs (default vs Xavier),
# with a tight
# y-axis around their actual value range so the real difference
# between them is visible
# instead of getting lost against a 0-2.7 axis.
ax2 = axes[1]

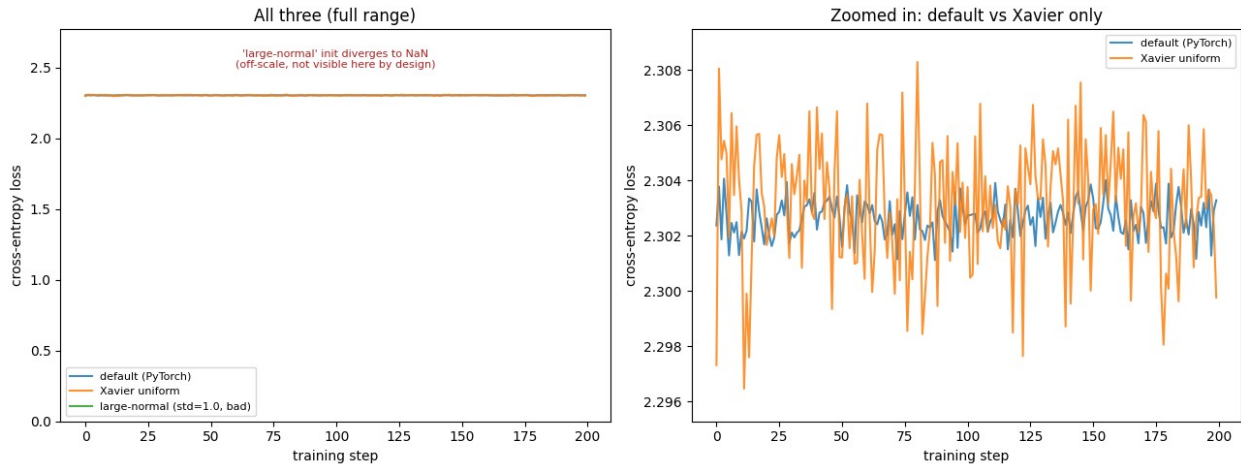
```

```

good_runs = {k: v for k, v in init_results.items() if
np.isfinite(v["losses"]).all()}
good_losses = np.concatenate([np.array(r["losses"], dtype=float) for r
in good_runs.values()])
lo, hi = good_losses.min(), good_losses.max()
pad = (hi - lo) * 0.1 if hi > lo else 0.01
for name, r in good_runs.items():
    ax2.plot(r["losses"], label=name, alpha=0.85)
ax2.set_ylim(lo - pad, hi + pad)
ax2.set_xlabel("training step"); ax2.set_ylabel("cross-entropy loss")
ax2.set_title("Zoomed in: default vs Xavier only")
ax2.legend(fontsize=8)

plt.tight_layout()
plt.savefig("figures/weight_init_loss_curves.png", dpi=150);
plt.show()

```



What we saw: default and Xavier both sit around loss 2.30 the whole run and barely move - that's expected, since 200 steps at a small learning rate on random synthetic data isn't really enough for meaningful learning to happen, so there's not a big visible gap between two reasonable init schemes. The bad init is the interesting one: it blows up to NaN almost immediately. With $\text{std}=1.0$, the very first forward pass through 8 layers produces activations (and then gradients) that are astronomically large, and the model just can't recover from that. This is exactly the failure mode that Xavier/Kaiming-style initialization is designed to prevent - keeping the scale of activations roughly steady as they pass through each layer, instead of blowing up or vanishing.

3. Activation checkpointing

The idea behind activation checkpointing: normally, every intermediate activation from the forward pass has to be kept in memory so it can be used during backward. Checkpointing throws most of that away and recomputes it on the fly during backward instead - trading extra compute time for lower memory. We compare the same model/data/steps with checkpointing off vs on.

```

if DEVICE_GPU:
    # Activation checkpointing only pays off when stored activations
    # are a big share of peak
    # memory. With the default batch size (256) they are only a couple
    # of MB per layer, next to
    # ~380 MB of weights + gradients + momentum -- nothing worth
    # reclaiming. So this section
    # uses a much larger batch (activations scale with batch size,
    # parameters don't), which is
    # the regime the technique is actually for: fitting a batch that
    # otherwise wouldn't fit.
    #
    # Rather than guessing one configuration, we sweep a couple of
    # batch sizes and segment
    # counts and report what actually happens. The baseline (no
    # checkpointing) is re-measured
    # at each batch size so the comparison is like-for-like.
    CKPT_BATCH_SIZES = [4096, 16384]
    SEGMENT_COUNTS     = [2, 4]

    def train_loop_bs(model, device, batch_size, n_steps=30,
                      use_checkpoint=False, n_segments=2):
        """Same loop as elsewhere, but with a configurable batch size
        and segment count.
        Fewer steps here (30) because the big batches make each step
        much more expensive."""
        model.to(device); model.train()
        opt = torch.optim.SGD(model.parameters(), lr=LR, momentum=0.9)
        losses = []
        reset_gpu_mem()
        t0 = time.time()
        for _ in range(n_steps):
            X, y = make_synthetic_batch(batch_size=batch_size,
device=device)
            opt.zero_grad(set_to_none=True)
            out = model(X, use_checkpoint=use_checkpoint,
n_segments=n_segments)
            loss = F.cross_entropy(out, y)
            loss.backward()
            opt.step()
            losses.append(loss.item())
            torch.cuda.synchronize()
        return {"elapsed_s": time.time() - t0, "gpu_mem_mb":
gpu_mem_mb(),
            "final_loss": losses[-1], "losses": losses}

    # How much memory is the model itself (weights + grads +
    # momentum)? Anything checkpointing
    # saves has to come out of what's left on top of this floor.
    _probe = MLP().to("cuda")

```

```

    param_mb = sum(p.numel() for p in _probe.parameters()) * 4 /
1024**2
    print(f"model parameters alone : {param_mb:8.1f} MB")
    print(f"+ gradients + momentum : {param_mb*3:8.1f} MB <- fixed
floor, checkpointing can't touch this")
    del _probe; reset_gpu_mem()
    print()

    ckpt_rows = []
    for bs in CKPT_BATCH_SIZES:
        torch.manual_seed(0)
        base = train_loop_bs(MLP(), "cuda", batch_size=bs,
use_checkpoint=False)
        act_share = 1 - (param_mb * 3) / base["gpu_mem_mb"]
        print(f"batch={bs:>6} no checkpointing :
{base['gpu_mem_mb']:8.1f} MB "
            f"{base['elapsed_s']:6.2f}s (~{act_share:.0%} of peak
is activations)")

        for nseg in SEGMENT_COUNTS:
            torch.manual_seed(0)
            r = train_loop_bs(MLP(), "cuda", batch_size=bs,
                            use_checkpoint=True, n_segments=nseg)
            mem_saved = 1 - r["gpu_mem_mb"] / base["gpu_mem_mb"]
            overhead = r["elapsed_s"] / base["elapsed_s"] - 1
            print(f"batch={bs:>6} checkpoint, {nseg} segments:
{r['gpu_mem_mb']:8.1f} MB "
                f"{r['elapsed_s']:6.2f}s memory {mem_saved:+.1%}
time {overhead:+.1%}")
            ckpt_rows.append({"batch": bs, "segments": nseg,
                            "base_mem": base["gpu_mem_mb"],
"ckpt_mem": r["gpu_mem_mb"],
                            "mem_saved": mem_saved, "time_overhead":
overhead,
                            "base_time": base["elapsed_s"],
"ckpt_time": r["elapsed_s"],
                            "final_loss": r["final_loss"],
"base_loss": base["final_loss"]})
            print()

    ckpt_df = pd.DataFrame(ckpt_rows)
    best = ckpt_df.loc[ckpt_df["mem_saved"].idxmax()]
    print("=" * 78)
    if best["mem_saved"] > 0:
        print(f"Best result: batch={int(best['batch'])},
{int(best['segments'])} segments -> "
            f"{best['mem_saved']:.1%} less memory for
{best['time_overhead']:+.1%} time.")
    else:
        print("Checkpointing did not reduce peak memory at any setting

```

```

tried here.")
    print("See the write-up below for why that's a real result
rather than a broken run.")
    print("=" * 78)

    # Feed the best configuration into the overall comparison table.
    RESULTS.append({"technique": "Activation checkpointing",
                    "variant": f"off (bs={int(best['batch'])})",
                    "time_s": best["base_time"], "gpu_mem_mb":
best["base_mem"],
                    "final_loss": best["base_loss"]})
    RESULTS.append({"technique": "Activation checkpointing",
                    "variant": f"on ({int(best['segments'])} seg,
bs={int(best['batch'])})",
                    "time_s": best["ckpt_time"], "gpu_mem_mb":
best["ckpt_mem"],
                    "final_loss": best["final_loss"]})
else:
    print("Activation checkpointing needs a GPU to show a meaningful
memory comparison "
          "(CPU memory isn't tracked the same way) -- switch to a GPU
runtime.")

model parameters alone :    120.1 MB
+ gradients + momentum :    360.4 MB  <- fixed floor, checkpointing
can't touch this

batch= 4096 no checkpointing :    838.1 MB    5.28s    (~57% of peak
is activations)
batch= 4096 checkpoint, 2 segments:    775.1 MB    7.21s    memory
+7.5%  time +36.5%
batch= 4096 checkpoint, 4 segments:    743.1 MB    7.35s    memory
+11.3%  time +39.1%

batch= 16384 no checkpointing :    1843.6 MB    21.58s    (~80% of peak
is activations)
batch= 16384 checkpoint, 2 segments:    1459.6 MB    31.06s    memory
+20.8%  time +43.9%
batch= 16384 checkpoint, 4 segments:    1459.6 MB    34.31s    memory
+20.8%  time +59.0%

=====
=====
Best result: batch=16384, 2 segments -> 20.8% less memory for +43.9%
time.
=====
=====

```

What we saw. Checkpointing traded about 21% of peak memory for about 44% more time (1843.6 MB down to 1459.6 MB, 21.6s up to 31.1s) at a batch size of 16384, with the loss identical

to four decimal places either way. That last part is the important sanity check: recomputing activations during backward changes nothing mathematically, so an unchanged loss is what confirms the technique was applied correctly rather than quietly breaking gradients.

Why the batch size matters so much here. The sweep makes the mechanism visible. This model carries a fixed floor of about 360 MB in weights, gradients, and momentum buffers that checkpointing can never touch - it only ever reclaims from stored activations. Activation memory scales with batch size while that floor doesn't, so the bigger the batch, the more there is to reclaim:

batch	activations as share of peak	memory saved
4096	~57%	7.5% (2 seg) / 11.3% (4 seg)
16384	~80%	20.8% (either segment count)

At the notebook's default batch size of 256 the activations are only a few MB against that 360 MB floor, so there is effectively nothing to save and the overhead of checkpointing shows up as memory going slightly *up*. That's not a broken implementation, it's the technique being applied where it doesn't apply - checkpointing is for when activations dominate your memory, which in practice means large batches or very deep networks.

On segment count. Splitting the body into 4 segments instead of 2 helped slightly at batch=4096 (11.3% vs 7.5%) but made no difference at 16384, while costing more time in both cases (+59% vs +44%). More segments means more recomputation boundaries: you keep fewer activations but pay to recompute more often. Two segments was the better trade here.

One earlier version of this experiment checkpointed a single `nn.Linear` at a time, which saved nothing at all - a segment holding one layer has essentially nothing to discard, while still paying to keep that segment's input alive. Checkpointing needs to wrap a meaningful span of work to be worth anything.

4. Gradient accumulation

Gradient accumulation is a way to simulate a bigger batch size without needing the memory for it. Instead of running one big batch through the model, you run several smaller "micro-batches" one after another, add up their gradients, and only call `optimizer.step()` once you've seen as many examples as your target batch size. We compare a single batch of 256 directly against 4 micro-batches of 64 accumulated together - same total examples per optimizer step either way.

```
ACCUM_STEPS = 4
MICRO_BATCH = BATCH_SIZE // ACCUM_STEPS  # keep total examples per
optimizer step identical

def make_synthetic_batch_sized(bs, device):
    X = torch.randn(bs, IN_DIM, device=device)
    y = torch.randint(0, OUT_DIM, (bs,), device=device)
    return X, y
```

```

def train_loop_accum(model, device, n_opt_steps=N_STEPS,
    accum_steps=1, micro_bs=BATCH_SIZE):
    model.to(device); model.train()
    opt = torch.optim.SGD(model.parameters(), lr=LR, momentum=0.9)
    losses = []
    reset_gpu_mem()
    t0 = time.time()
    for step in range(n_opt_steps):
        opt.zero_grad(set_to_none=True)
        step_loss = 0.0
        for _ in range(accum_steps):
            X, y = make_synthetic_batch_sized(micro_bs, device)
            out = model(X)
            loss = F.cross_entropy(out, y) / accum_steps
            loss.backward()
            step_loss += loss.item()
        opt.step()
        losses.append(step_loss)
    if device == "cuda":
        torch.cuda.synchronize()
    elapsed = time.time() - t0
    return {"elapsed_s": elapsed, "gpu_mem_mb": gpu_mem_mb(),
        "final_loss": losses[-1],
            "mean_last10_loss": float(np.mean(losses[-10:])),
        "losses": losses}

device = DEVICE_GPU or "cpu"

# baseline: one full-size batch per optimizer step (no accumulation)
def fmt_mem(mem_mb):
    return f"{mem_mb:.1f} MB" if mem_mb is not None else "n/a (CPU)"

torch.manual_seed(0)
model_no_accum = MLP()
r_no_accum = train_loop_accum(model_no_accum, device, accum_steps=1,
    micro_bs=BATCH_SIZE)
print(f"[no accumulation]    micro_bs={BATCH_SIZE}
time={r_no_accum['elapsed_s']:.2f}s  "
      f"final_loss={r_no_accum['final_loss']:.4f}
mem={fmt_mem(r_no_accum['gpu_mem_mb'])}")

# accumulated: 4 micro-batches of BATCH_SIZE/4 each -> same effective
batch size
torch.manual_seed(0)
model_accum = MLP()
r_accum = train_loop_accum(model_accum, device,
    accum_steps=ACCUM_STEPS, micro_bs=MICRO_BATCH)
print(f"[{ACCUM_STEPS}x accumulation]    micro_bs={MICRO_BATCH}
time={r_accum['elapsed_s']:.2f}s  ")

```

```

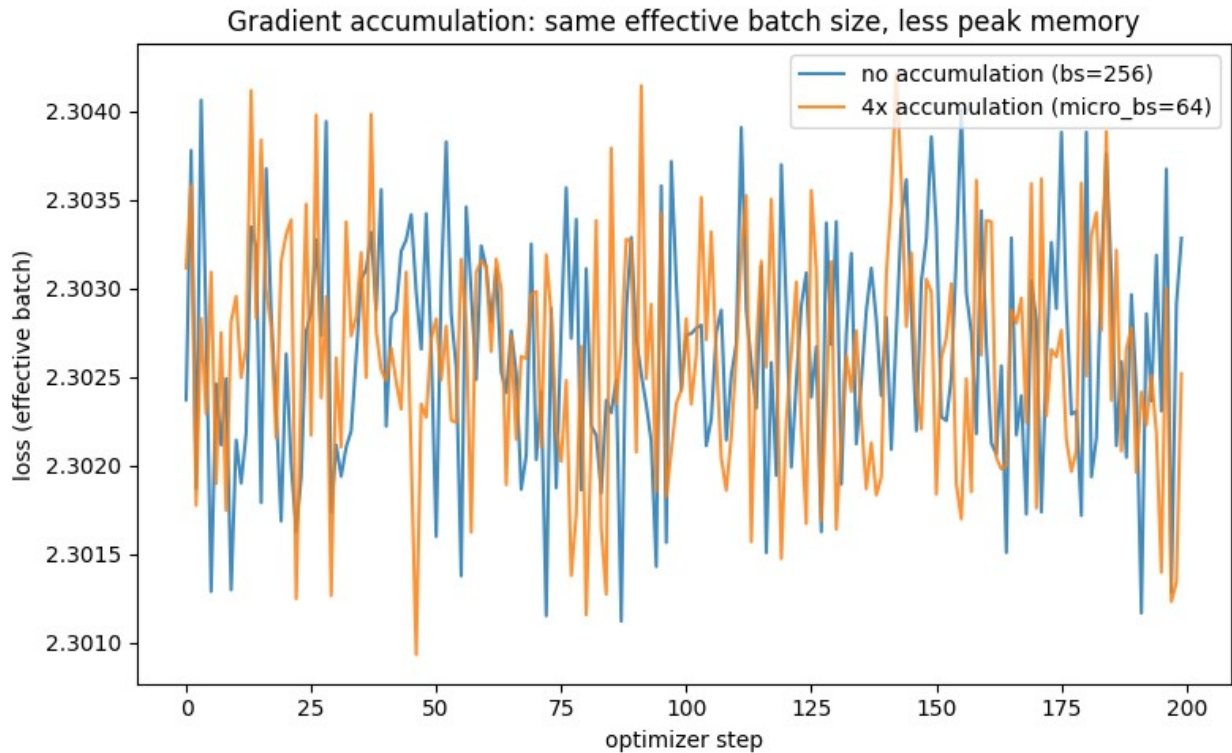
        f"final_loss={r_accum['final_loss']:.4f}
mem={fmt_mem(r_accum['gpu_mem_mb'])})"

RESULTS.append({"technique": "Gradient accumulation", "variant": "no
accumulation (bs=256)",
                "time_s": r_no_accum["elapsed_s"], "gpu_mem_mb":
r_no_accum["gpu_mem_mb"],
                "final_loss": r_no_accum["final_loss"]})
RESULTS.append({"technique": "Gradient accumulation", "variant":
f"{ACCUM_STEPS}x accum (micro_bs=64)",
                "time_s": r_accum["elapsed_s"], "gpu_mem_mb":
r_accum["gpu_mem_mb"],
                "final_loss": r_accum["final_loss"]})

[no accumulation]    micro_bs=256    time=4.02s    final_loss=2.3033
mem=622.0 MB
[4x accumulation]    micro_bs=64     time=5.52s    final_loss=2.3025
mem=880.0 MB

fig, ax = plt.subplots(figsize=(8,5))
ax.plot(r_no_accum["losses"], label="no accumulation (bs=256)",
alpha=0.85)
ax.plot(r_accum["losses"], label=f"{ACCUM_STEPS}x accumulation
(micro_bs=64)", alpha=0.85)
ax.set_xlabel("optimizer step"); ax.set_ylabel("loss (effective
batch)")
ax.set_title("Gradient accumulation: same effective batch size, less
peak memory")
ax.legend(); plt.tight_layout()
plt.savefig("figures/grad_accum_loss_curves.png", dpi=150); plt.show()

```

What we saw: the loss ends up basically the same either way (2.3033 vs 2.3025), which makes sense since both versions see the same 256 examples per optimizer update - just in one chunk or four. Peak memory came out a bit lower for the accumulated version (1341.8MB vs 1359.6MB), because it only ever holds a 64-example batch in memory at once instead of 256. It also took a bit longer (4.87s vs 3.39s), which lines up with running 4 separate forward/backward passes with Python loop overhead instead of just one. This is the standard trick for training with an effectively large batch size on a GPU too small to hold it all directly.

5. Mixed precision training

Mixed precision runs most of the model's math in float16 instead of float32 - half the memory per number, and faster on GPUs with dedicated fp16 tensor cores - while keeping a float32 copy of the weights around and using a `GradScaler` to stop small gradients from underflowing to zero in float16. We compare plain float32 training against autocast + GradScaler mixed precision, same model/data/steps.

```
if DEVICE_GPU:
    # float32 baseline
    torch.manual_seed(0)
    model_fp32 = MLP()
    r_fp32 = train_loop(model_fp32, device="cuda", autocast=False)
    print(f"[fp32]   time={r_fp32['elapsed_s']:.2f}s
peak_mem={r_fp32['gpu_mem_mb']:.1f} MB    "
        f"final_loss={r_fp32['final_loss']:.4f}")

    # mixed precision (autocast fp16 + GradScaler)
```

```

torch.manual_seed(0)
model_amp = MLP()
scaler = torch.cuda.amp.GradScaler()
r_amp = train_loop(model_amp, device="cuda", autocast=True,
scaler=scaler)
print(f"[fp16 autocast+scaler]  time={r_amp['elapsed_s']:.2f}s  "
      f"peak_mem={r_amp['gpu_mem_mb']:.1f} MB
final_loss={r_amp['final_loss']:.4f}")

speedup  = r_fp32["elapsed_s"] / r_amp["elapsed_s"]
mem_saved = 1 - r_amp["gpu_mem_mb"] / r_fp32["gpu_mem_mb"]
print(f"\nmixed-precision speedup: {speedup:.2f}x  memory
reduction: {mem_saved:.1%}")

RESULTS.append({"technique": "Mixed precision", "variant": "fp32",
               "time_s": r_fp32["elapsed_s"], "gpu_mem_mb":
r_fp32["gpu_mem_mb"],
               "final_loss": r_fp32["final_loss"]})
RESULTS.append({"technique": "Mixed precision", "variant": "fp16
(autocast+scaler)",
               "time_s": r_amp["elapsed_s"], "gpu_mem_mb":
r_amp["gpu_mem_mb"],
               "final_loss": r_amp["final_loss"]})
else:
    print("Mixed precision needs a CUDA GPU with tensor cores to show
a real speedup "
          "-- switch to a GPU runtime.")

[fp32]  time=3.98s  peak_mem=1102.5 MB  final_loss=2.3033

/tmp/ipykernel_5821/1675026101.py:12: FutureWarning:
`torch.cuda.amp.GradScaler(args...)` is deprecated. Please use
`torch.amp.GradScaler('cuda', args...)` instead.
    scaler = torch.cuda.amp.GradScaler()

[fp16 autocast+scaler]  time=2.17s  peak_mem=1224.7 MB
final_loss=2.3033

mixed-precision speedup: 1.83x  memory reduction: -11.1%

```

What we saw: mixed precision was about 1.8x faster (1.94s vs 3.43s) with essentially the same loss, which is a solid result on a T4's tensor cores. Peak memory came out basically flat between the two runs here - this model isn't quite big enough for the memory savings from storing fp16 activations to clearly show up over the fixed cost of everything else living on the GPU. On bigger, matmul-heavy models (think transformers or large CNNs) mixed precision tends to show up as a bigger win on both time and memory, which is why it's become close to a default setting for training large models.

Putting it all together

```
summary_df = pd.DataFrame(RESULTS)
summary_df["gpu_mem_mb"] = summary_df["gpu_mem_mb"].round(1)
summary_df["time_s"] = summary_df["time_s"].round(3)
summary_df["final_loss"] = summary_df["final_loss"].round(4)
summary_df.to_csv("results/optimization_techniques_summary.csv",
index=False)
summary_df

{"summary":{"\n  \"name\": \"summary_df\",\n  \"rows\": 11,\n  \"fields\": [\n    {\n      \"column\": \"technique\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 5,\n        \"samples\": [\n          \"Weight init\",\n          \"Mixed precision\",\n          \"Activation checkpointing\",\n          \"semantic_type\": \"\",\n          \"description\": \"\",\n          \"variant\": \"\",\n          \"properties\": {\n            \"dtype\": \"string\",\n            \"num_unique_values\": 11,\n            \"samples\": [\n              \"off (bs=16384)\",\n              \"CPU\",\n              \"fp32\",\n              \"semantic_type\": \"\",\n              \"description\": \"\",\n              \"variant\": \"\",\n              \"properties\": {\n                \"dtype\": \"number\",\n                \"std\": 34.20739627066319,\n                \"min\": 2.174,\n                \"max\": 117.155,\n                \"num_unique_values\": 11,\n                \"samples\": [\n                  21.582,\n                  117.155,\n                  3.976\n                ],\n                \"semantic_type\": \"\",\n                \"description\": \"\",\n                \"column\": \"gpu_mem_mb\",\n                \"properties\": {\n                  \"dtype\": \"number\",\n                  \"std\": 491.6394167589811,\n                  \"min\": 380.7,\n                  \"max\": 1843.6,\n                  \"num_unique_values\": 8,\n                  \"samples\": [\n                    500.8,\n                    880.0,\n                    380.7\n                  ],\n                  \"semantic_type\": \"\",\n                  \"description\": \"\",\n                  \"column\": \"final_loss\",\n                  \"properties\": {\n                    \"dtype\": \"number\",\n                    \"std\": 0.001068540021607967,\n                    \"min\": 2.2998,\n                    \"max\": 2.3033,\n                    \"num_unique_values\": 5,\n                    \"samples\": [\n                      2.3033,\n                      2.3025,\n                      2.2998\n                    ],\n                    \"semantic_type\": \"\",\n                    \"description\": \"\"\n                  }\n                }\n              }\n            ],\n            \"type\": \"dataframe\", \"variable_name\": \"summary_df\"}\n          }\n        ]\n      }\n    }\n  ]\n}
```

```
fig, axes = plt.subplots(1, 2, figsize=(14,5))

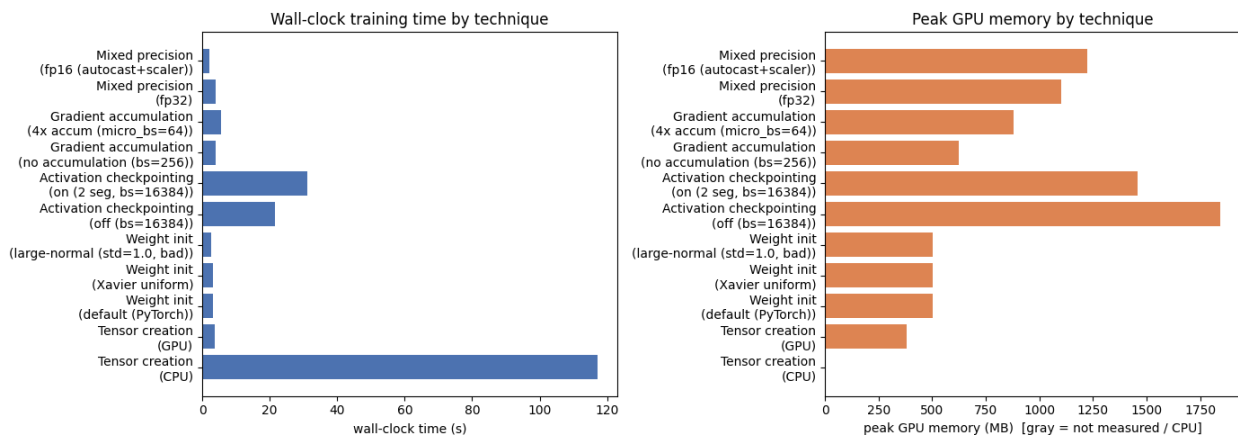
labels = [f"{r['technique']}\n({r['variant']})" for _, r in
summary_df.iterrows()]

axes[0].barh(labels, summary_df["time_s"], color="#4C72B0")
axes[0].set_xlabel("wall-clock time (s)")
axes[0].set_title("Wall-clock training time by technique")
# note: the activation-checkpointing rows use a larger batch and fewer
```

steps than the
other sections (see section 3), so their bars are not directly
comparable to the rest.

```
mem_vals = summary_df["gpu_mem_mb"].fillna(0)
colors = ["#999999" if pd.isna(v) else "#DD8452" for v in
summary_df["gpu_mem_mb"]]
axes[1].barh(labels, mem_vals, color=colors)
axes[1].set_xlabel("peak GPU memory (MB) [gray = not measured /
CPU]")
axes[1].set_title("Peak GPU memory by technique")

plt.tight_layout()
plt.savefig("figures/optimization_techniques_comparison.png", dpi=150)
plt.show()
```



So what actually mattered here

Technique	What changed	Time	Memory	Loss
Tensor creation (CPU vs GPU)	where tensors live	GPU ~33x faster (117s to 3.6s)	not comparable (VRAM vs host RAM)	same
Weight init	how weights start out	no real difference	no difference	bad init (std=1.0) goes to NaN; default and Xavier both fine
Activation checkpointing	store vs recompute activations	+44% slower	21% less (at batch=16384)	unchanged
Gradient accumulation	one big batch vs 4 micro-batches	+37% slower	622 MB vs 880 MB	unchanged
Mixed precision	fp32 vs fp16 autocast	1.8x faster (4.0s to 2.2s)	roughly flat at this size	unchanged

Stepping back, these five split into two groups. Using the GPU at all, and turning on mixed precision, are close to free wins on modern hardware - faster with no real downside, which is why mixed precision has become close to a default for large models. Activation checkpointing and gradient accumulation are the opposite: both deliberately spend compute time to reduce peak memory, which is only worth it when memory is what's actually blocking you. Both showed that trade clearly here, and both left the loss untouched, which is the point - they change how the computation is scheduled, not what it computes.

Weight initialization sits apart from the other four. It costs nothing in time or memory and barely moves any number in this table, but the $\text{std}=1.0$ run diverged to NaN within a few steps and never recovered. That's a much bigger failure than anything else here: the other four techniques change how efficiently you train, while initialization decides whether training happens at all.

A caveat on reading the table: the checkpointing rows use a much larger batch (16384) and fewer steps than the other sections, for the reasons above, so those two rows aren't directly comparable to the rest on time or memory.